JS

FRONTEND PRACTICE
HTML · CSS · JavaScript
for the Accenture frontend round

Frontend Practice,
Explained Simply

14 exercises in 10 days, 2 hours a day, from **dexterhere04/accenture-frontend-practice**. Written for a beginner: every idea explained right before you need it, a plan and three hints for each exercise, then a solution tested against the repository's own grader, the mistakes beginners make, and the questions interviewers ask.

the pattern behind every exercise

```
// 1. select
const btn = document.getElementById('increment');

// 2. listen
btn.addEventListener('click', () => {
  // 3. change
  count = count + 1;
  countEl.textContent = count;
});
```

Labels & inputs · Box model · Flexbox · Grid · Centering · Events · preventDefault · if / else · Arrays · filter & sort · createElement · classList · CSS variables · Regex · data-* · Modulo · Date & setInterval · State + render · Mock round

Every solution in this guide passes the exercise's Playwright tests (57 of 57 across the 14 exercises). Outputs shown under examples come from running them. Exercise screenshots were taken from the starter files and from the tested solutions.

How to Use This Guide

This guide takes you through **14 of the 20 exercises** in dexterhere04/accenture-frontend-practice in **10 days of 2 hours**. It assumes you know very little HTML, CSS or JavaScript. Every idea is explained the first time you need it, right before the exercise that uses it.

Each exercise follows the same steps:

1. **What you are building:** a picture of the starter page next to the finished one, and the list of things the grader checks.
2. **New ideas:** only what this exercise needs, with tiny examples and their real output.
3. **Read the starter files:** what is already written, and the ids and classes you will use.
4. **The plan in plain English:** the steps before any code.
5. **Hints:** three, from a gentle push to almost the answer. Read one at a time.
6. **The solution, line by line,** to read **after** you have tried.
7. **Mistakes beginners make, interview questions,** and a **practice twist** to do on your own.

> 📝 **What was checked**
>
> Every solution in this guide was run against the repository's own grader (its Playwright tests): **all 57 tests of the 14 exercises pass**. The outputs printed under the small examples come from running them. The scripts that did this are in `frontend-practice-source/verify/`.

The 10-day plan



Figure 0.1 *Fourteen exercises in nine days, then a mock round on Day 10. The two stars mark the questions that candidates reported from the real Accenture round.*

Day	Exercises	New ideas
0 · evening before, 45 min	Setup	Node.js, VS Code, cloning the repo, running the grader
1	03 Click Counter	variables, functions, the **select → listen → change** pattern
2	01 Fix the Form · 08 Show / Hide Password	labels and input types, form submit, `if` / `else`
3	02 Profile Card · 04 Center It	the box model, flexbox, grid, absolute positioning
4	05 Search Box ⭐	arrays, `filter`, creating elements, the **render function**
5	06 To-Do List · 09 Dark Mode	building elements, `classList`, CSS variables
6	07 Form Validation	`else if`, regular expressions, a “valid” flag
7	13 Accordion · 14 Tab Switcher	`querySelectorAll`, `forEach`, `data-*`, **all off, one on**
8	16 Carousel · 17 Live Clock ⭐	an index with wrap-around, `Date`, `setInterval`
9	15 Dynamic Table	objects, filter + sort, **state + render**
10	Mock round + revision	timed redo of 05 and 17, cheat sheets, interview questions

Why these 14? The real round gives you pre-built files and asks you to change them, with DOM changes, events and dynamic lists at the centre. The 14 cover all of that, plus the HTML and CSS basics you need to read any starter file. The other six (10, 11, 12, 18, 19, 20) are good practice but either mostly CSS or need ideas (fetching data, saving to the browser) that are easier once these 14 feel normal. They are in the **Bonus** chapter for after Day 10.

Your 2 hours, every day

Minutes	What you do
0–25	Read the day’s **New ideas**. Type every small example into the browser console (Day 0 shows how).
25–35	Read `task.md` and the starter files. Write the plan as comments at the top of `script.js`.
35–90	**Build it.** Stuck for 10 minutes? Read Hint 1. Still stuck after 10 more? Hint 2.
90–100	Run the grader. Fix what fails.
100–120	Read the solution explanation and compare it with yours. Answer the interview questions **out loud**. Do the practice twist.

On days with two exercises, give the first one about 70 minutes and the second about 50.

📌 **The five rules**

1. **Type the code, never paste it.** Your fingers learn too.
2. **Try for 10 minutes before every hint.** Being stuck is where the learning happens.
3. **Run the grader after every exercise.** Green means it really works.
4. **Explain your code out loud** as if to an interviewer. If you can't explain a line, read its explanation again.
5. **Hard exercise? Redo it the next morning from the starter files,** without looking. That is how confidence is built.

Progress tracker

Tick each box as you go. "Redone from blank" means you deleted your code and wrote it again without help.

Ex	Name	Day	Tests pass	Redone from blank	Confidence (1–5)
03	Click Counter	1	☐	☐	
01	Fix the Form	2	☐	☐	
08	Show / Hide Password	2	☐	☐	
02	Style the Profile Card	3	☐	☐	
04	Center It	3	☐	☐	
05	E-commerce Search Box ⭐	4	☐	☐	
06	To-Do List	5	☐	☐	
09	Dark-Mode Toggle	5	☐	☐	
07	Form Validation	6	☐	☐	
13	Accordion	7	☐	☐	
14	Tab Switcher	7	☐	☐	
16	Image Carousel	8	☐	☐	
17	Live Digital Clock ⭐	8	☐	☐	
15	Dynamic Table	9	☐	☐	

Contents

The 10-day plan 2
Your 2 hours, every day 3
Progress tracker 4

DAY 0 Start Here: Setup and the Big Picture 7
0.1 What the real task looks like 7
0.2 Install the tools 7
0.3 Get the exercises 8
0.4 Open an exercise in the browser 9
0.5 Run the grader 9
0.6 How a web page works: three files, three jobs 10
0.7 The DOM: the page as a tree 12
0.8 The browser console: where you test and debug 12
0.9 The six moves you will use in every exercise 14

DAY 1 Your First JavaScript: the Click Counter 15
Exercise 03 · Click Counter 15

DAY 2 Forms and Toggles 23
Exercise 01 · Fix the Form 23
Exercise 08 · Show / Hide Password 29

DAY 3 CSS Day: Boxes, Rows and Centering 35
Exercise 02 · Style the Profile Card 35
Exercise 04 · Center It 41

DAY 4 The Real Question: E-commerce Search Box 45
Exercise 05 · E-commerce Search Box ⭐ 45

DAY 5 Building Elements and Switching Classes 54
Exercise 06 · To-Do List 54
Exercise 09 · Dark-Mode Toggle 61

DAY 6 Checking What the User Typed: Form Validation 65
Exercise 07 · Form Validation 66

DAY 7 One Open at a Time: Accordion and Tabs 73
Exercise 13 · Accordion 73
Exercise 14 · Tab Switcher 78

DAY 8 Counting in Circles, and Time 84
Exercise 16 · Image Carousel 84
Exercise 17 · Live Digital Clock ⭐ 90

DAY 9 Data on Screen: the Dynamic Table 95
Exercise 15 · Dynamic Table 95

DAY 10 Mock Round and Revision 104
10.1 The mock round 104
10.2 The three patterns 106
10.3 Cheat sheets 107
10.4 When something doesn't work 109
10.5 Rapid-fire interview questions 109
10.6 After Day 10 112

BONUS The Other Six Exercises 113
Exercise 10 · Responsive Navbar 113

Exercise 11 · Responsive Card Grid 114
Exercise 12 · Progress Bar + Spinner 114
Exercise 19 · Persistent Notes 114
Exercise 20 · Quiz 115
Exercise 18 · Fetch & Render 116

DAY 0 45 minutes, the evening before Day 1

JS

Start Here: Setup and the Big Picture

TODAY YOU WILL LEARN

- What the Accenture frontend task looks like
- **Installing the tools** and getting the 20 exercises onto your laptop
- Opening an exercise in the browser, and running the **grader**
- How a web page works: **HTML, CSS and JavaScript** in one picture
- The **DOM**: the page as a tree that JavaScript can change
- The browser **console**, your best friend for finding mistakes

0.1 What the real task looks like

From the candidate reports collected in the repository's README:

- The coding round is about **60 minutes for 3 tasks** (SQL, backend and frontend). The frontend task is the **easy** one.
- You do **not** start from an empty file. You get an `index.html`, a `style.css` and a `script.js` that already exist, and you **change** them: *"Task 1: Modify the HTML file. Task 2: Make changes in the CSS file. Task 3: Update logic in the JavaScript file."*
- What it tests: **finding elements on the page, reacting to clicks and typing, and changing what the page shows**. No React, no frameworks. Plain JavaScript.

Every exercise in this repository is built the same way, so practising here is practising the real thing.

0.2 Install the tools

You need three programs. All are free.

Tool	What it is for	Where to get it
Node.js (the LTS version)	runs the grader; also gives you `npm`	nodejs.org
VS Code	the editor you write code in	code.visualstudio.com
Git	downloads the exercises	git-scm.com (on Windows, keep the default options; it also installs **Git Bash**)

After installing, open a terminal (on Windows: **Git Bash** or **PowerShell**; on a Mac: **Terminal**) and check:

TERMINAL check that the tools are installed

```
node --version
npm --version
git --version
```

Each command should print a version number, such as `v22.x.x`. If one says *command not found*, close the terminal, open a new one and try again. If it still fails, install that tool again.

0.3 Get the exercises

Pick a folder for your work (for example `Documents`), then run these four commands one by one:

TERMINAL download the exercises and install the grader

```
git clone https://github.com/dexterhere04/accenture-frontend-practice.git
cd accenture-frontend-practice
npm install
npx playwright install chromium
```

What each one does:

1. `git clone` downloads the repository into a new folder.
2. `cd` (*change directory*) moves the terminal into that folder.
3. `npm install` downloads **Playwright**, the tool that tests your pages.
4. `npx playwright install chromium` downloads the browser Playwright uses. It is more than 100 MB, so wait for it to finish.

Now open the folder in VS Code (**File → Open Folder...**). You will see `exercises/` with 20 folders. Each one has:

File	What it is
`task.md`	the question, written like the real assessment
`index.html`, `style.css`, `script.js`	**your** files, with `TODO` comments where you must write code
`test.spec.js`	the grader for this exercise (read it, don't change it)
`solution/`	the reference answer (don't open it before you've tried!)

✅ **Save your progress with Git (optional, but useful)**

After finishing an exercise, run `git add -A` and then `git commit -m "Finish exercise 03"`. Git keeps a copy of every finished version, so you can never lose working code.

0.4 Open an exercise in the browser

There are two easy ways:

- **Double-click** `exercises/03-click-counter/index.html` in your file explorer. It opens in your browser. This works for all 14 exercises in this guide.
- Or run `npm run dashboard` in the terminal and open **http://localhost:4173**. You get a page listing all 20 exercises with links. Stop the server with **Ctrl + C**.

After you change a file, **save it** (Ctrl + S) and **refresh** the browser (F5). The browser does not notice changes on its own.

0.5 Run the grader

The grader opens your page in a hidden browser, clicks and types like a user, and checks the result. Run it from the repository folder:

TERMINAL grade one exercise

```
npm test -- 03-click-counter
```

The part after `--` is the exercise folder's name. You can type just the start of it, but make sure it matches only one folder. Here is the **real** output for the untouched starter files of exercise 03 (lines shortened to fit the page):

OUTPUT npm test -- 03-click-counter, before writing any code

```
Running 4 tests using 2 workers

  ✓  1 … › starts at 0 (259ms)
  ✘  2 … › increments on each click (5.5s)
  ✘  3 … › reset returns the count to 0 (5.3s)
  ✘  4 … › the secondary button is visually distinct (291ms)

  1) … › increments on each click

    Error: expect(locator).toHaveText(expected) failed

    Locator:  locator('#count')
    Expected: "3" ← the test clicked 3 times
    Received: "0" ← but the page still shows 0
```

How to read it:

- ✓ means a check passed, ✘ means it failed. The first test passes already because the HTML shows `0` before you write any code.
- For each failure, Playwright tells you **which element** it looked at (`#count`), what it **expected** and what it **received**. That is usually enough to know what to fix.

After you finish the exercise, the same command prints:

OUTPUT npm test -- 03-click-counter, after the solution

```
✓  1 … › starts at 0 (329ms)
✓  2 … › increments on each click (401ms)
✓  3 … › reset returns the count to 0 (343ms)
✓  4 … › the secondary button is visually distinct (191ms)

4 passed (1.7s)
```

⚠️ **Windows:** `npm run test:solutions` **does not work in PowerShell or cmd**

That script starts with `SOLUTION=1`, which only works in Git Bash, macOS and Linux. To see the reference solution pass on Windows, either use **Git Bash**, or in **PowerShell** type:

```
$env:SOLUTION="1"; npx playwright test 03-click-counter; Remove-Item Env:SOLUTION
```

You will rarely need this. It is only a way to check that the grader itself works.

0.6 How a web page works: three files, three jobs

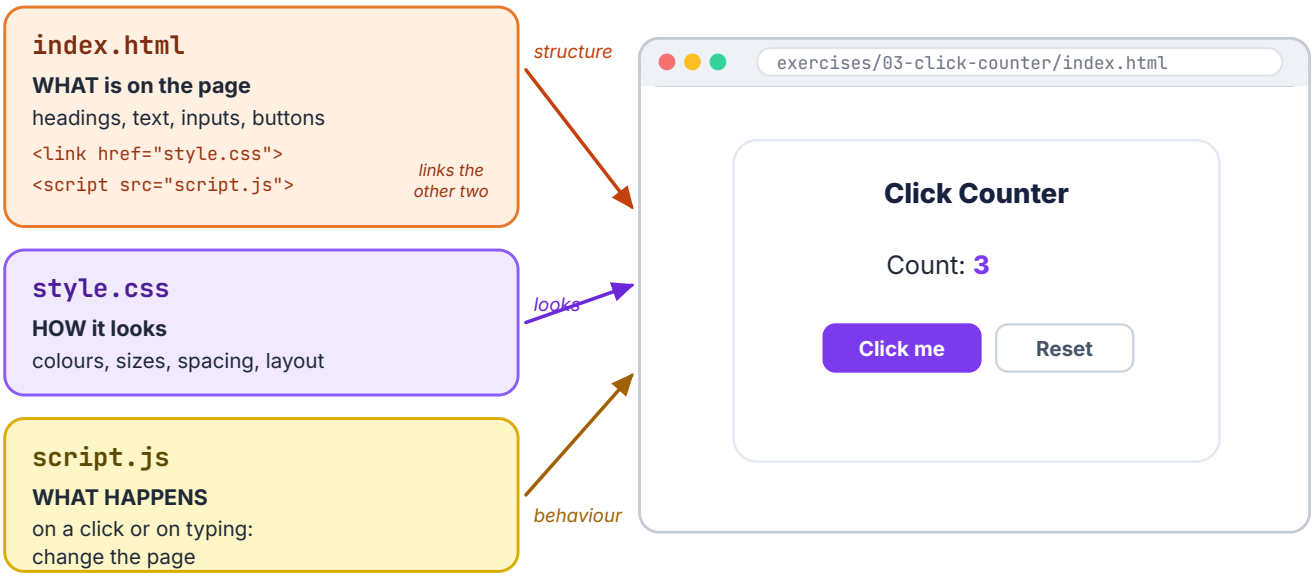


Figure 0.1 `index.html` *says* ***what is on the page***, `style.css` *says* ***how it looks***, *and* `script.js` *says* ***what happens*** *when the user does something. The HTML file links the other two.*

Think of a page as a person: **HTML** is the skeleton, **CSS** is the clothes, and **JavaScript** is the muscles that move. Let's read one line of each.

HTML: elements

HTML one HTML element

```html
<button id="increment" class="primary" type="button">Click me</button>
```

Piece	Name	Meaning
`<button>` … `</button>`	**opening and closing tag**	this is a button; everything between the tags is inside it
`id="increment"`	**attribute** `id`	a unique name. **Only one** element on the page may use it.
`class="primary"`	**attribute** `class`	a group name. **Many** elements can share a class, and one element can have several, separated by spaces: `class="tab active"`
`type="button"`	**attribute** `type`	what kind of button
`Click me`	**content**	the text the user sees

Elements live **inside** other elements, like boxes inside boxes. A `<section>` can hold an `<h1>`, a `<p>` and a `<div>`; the `<div>` can hold two buttons. Some elements have **no closing tag** because they can't contain anything, for example `<input />` and `<img />`.

CSS: rules

CSS one CSS rule

```css
.secondary {
  background: #ffffff;
  border: 1px solid #cbd5e1;
}
```

A rule is a **selector** (which elements?) followed by `{ property: value; }` pairs (how should they look?). The selectors you will use most:

Selector	Selects	Example
`button`	every `<button>`	`button { cursor: pointer; }`
`.secondary`	every element with **class** `secondary` (note the dot)	
`#count`	the element with **id** `count` (note the hash)	
`.card h1`	every `<h1>` **inside** an element with class `card` (space = inside)	
`.tab.active`	elements that have **both** classes (no space)	
`button:hover`	a button **while the mouse is over it**	

JavaScript: statements

JAVASCRIPT one line of JavaScript

```javascript
document.getElementById('count').textContent = 5;
```

Read it from left to right: *"In the page (`document`), get the element whose id is `count`, and set its text to 5."* The page now shows 5. That one line is what every exercise is about: **find something on the page, then change it**.

0.7 The DOM: the page as a tree

When the browser reads `index.html`, it builds a tree of objects in memory, one object per element. This tree is called the **DOM** (Document Object Model). JavaScript never edits the HTML file; it edits the **DOM**, and the browser redraws the page immediately.

index.html (text)

```
<section class="card">
  <h1>Click Counter</h1>
  <p class="count-row">
    Count:
    <span id="count">0</span>
  </p>
  <div class="actions">
    <button id="increment">
    <button id="reset">
  </div>
</section>
```

JavaScript never edits the file.
It changes the objects on the right,
and the browser redraws at once.

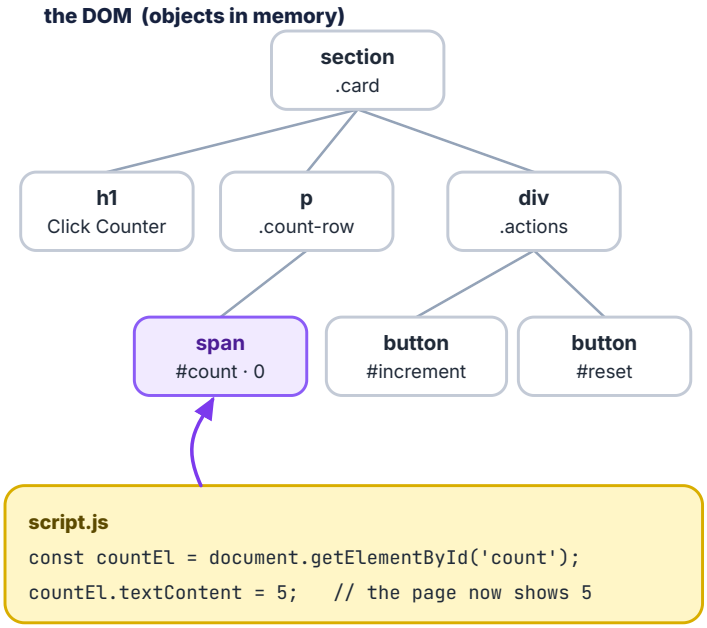


Figure 0.2 *The click counter's HTML (left) becomes a tree of objects (right). `getElementById('count')` hands your code the `<span>` object; changing its `textContent` changes what the user sees.*

> 💡 **The DOM is a live puppet**
>
> The HTML file is the **instruction sheet** for building the puppet. Once it is built, JavaScript holds the strings: pull one (`textContent = 5`) and the puppet moves right away. Nobody rewrites the instruction sheet.

0.8 The browser console: where you test and debug

Open any page in Chrome or Edge and press **F12** (on a Mac: **Cmd + Option + J**). Click the **Console** tab. Two uses:

1. Try code. Type a line and press Enter. The answer appears below it.

JAVASCRIPT type these into the console of the click-counter page

```
2 + 3
document.getElementById('count')
document.getElementById('count').textContent = 99
```

The third line changes the page right in front of you. **Every small example in this guide can be typed into the console like this.**

2. See what your code is doing. Put `console.log(...)` in `script.js` to print a value:

JAVASCRIPT printing from your script

```
console.log('script loaded');
console.log('count is now', count);
```

If something doesn't work, open the console first. **Red text is an error message**, with the file and line number. This is the one you will meet most:

OUTPUT the most common beginner error

```
Uncaught TypeError: Cannot read properties of null (reading 'addEventListener')
    at script.js:5:13 ← file : line : column
```

It means `getElementById` returned `null`: no element has that id. Almost always the id is misspelled (`'Count'` instead of `'count'`, or `'#count'` with a hash). Compare it letter by letter with `index.html`.

✅ **Why the `<script>` tag is at the bottom**

Every exercise puts `<script src="script.js"></script>` **just before** `</body>`. The browser reads HTML from top to bottom, so by the time it runs your script, every element above already exists and `getElementById` can find it. If the script were at the top, the elements wouldn't exist yet and you'd get the `null` error above.

0.9 The six moves you will use in every exercise

You don't need to learn all of JavaScript. These six moves solve every exercise in this guide. Each one is explained properly on the day you first need it.

#	Move	Code	First used
1	**Select** an element	`document.getElementById('count')`, `document.querySelectorAll('.tab')`	Day 1
2	**Listen** for something	`button.addEventListener('click', () => { ... })`	Day 1
3	**Read** what the user typed	`input.value`	Day 2
4	**Change** the page	`el.textContent = ...`, `el.classList.add('open')`, `el.hidden = true`	Day 1
5	**Create** new elements	`document.createElement('li')`, `list.appendChild(li)`	Day 4
6	**Loop** over a list	`array.forEach(...)`, `array.filter(...)`	Day 4

🎯 Day 0 check

1. Which file decides **what happens** when a button is clicked?
2. What is the difference between `id` and `class`?
3. In CSS, what does `.card h1` select? And `#count`?
4. You see `Cannot read properties of null (reading 'addEventListener')`. What do you check first?
5. You changed `script.js` but the browser shows the old behaviour. What did you forget?

✔️ Answers

1. `script.js` (JavaScript).
2. An `id` is unique (one element per page); a `class` can be shared by many elements, and one element can have several classes.
3. Every `<h1>` inside an element with class `card`; the element with id `count`.
4. The id inside `getElementById(...)`: compare its spelling with `index.html`, and don't put a `#` in it.
5. Saving the file (Ctrl + S) or refreshing the page (F5).

DAY 1 Exercise 03 · variables · functions · click events

JS

Your First JavaScript: the Click Counter

TODAY YOU WILL LEARN

- **Variables:** `let` and `const`, boxes with names
- **Functions:** a recipe you write once and run many times
- The pattern behind every exercise: **select → listen → change**
- CSS: styling a second button, and **which rule wins** when two rules disagree

💻 **Today's plan (2 hours)**

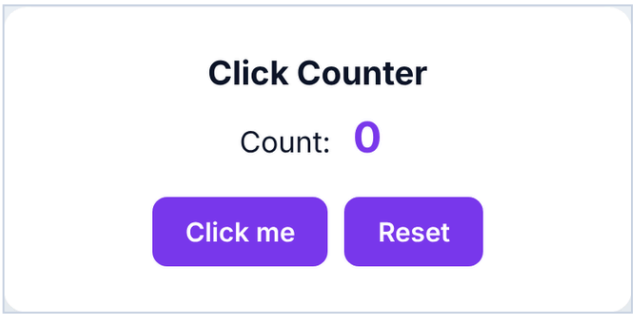
Setup should be done (Day 0). If it isn't, do it first and finish this exercise tomorrow morning.

0–30 New ideas · **30–40** read the starter files · **40–90** build · **90–100** grade · **100–120** solution, mistakes, interview questions, twist

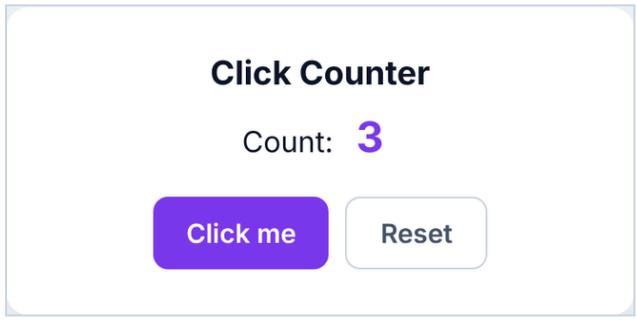
Exercise 03 · Click Counter

What you are building

START · the starter files



GOAL · after your code



The HTML and most of the CSS are done. You make the two buttons work and give **Reset** its own look. The grader checks four things:

Test	What it does
starts at 0	opens the page; `#count` must show `0`
increments on each click	clicks **Click me** 3 times; `#count` must show `3`
reset returns the count to 0	clicks twice (expects `2`), clicks **Reset**, expects `0`
the secondary button is visually distinct	the background of `.secondary` must **not** be the purple of the main button

New ideas

1. Variables: boxes with names

A **variable** is a named box that holds a value. You create one with `let` or `const`:

JAVASCRIPT variables

```
let count = 0;
count = count + 1;
count = count + 1;
console.log(count);

const greeting = 'Hello';
console.log(greeting + ', Ada');
console.log(typeof count, typeof greeting);
```

OUTPUT

```
2
Hello, Ada
number string
```

- `let` makes a box whose value **can change later** (`count` goes 0 → 1 → 2).
- `const` makes a box that **can never be given a new value**. Try `greeting = 'Hi'` and you get *TypeError: Assignment to constant variable.*
- Text goes inside quotes: `'Hello'` or `"Hello"`. Text is called a **string**.
- `count = count + 1` means: *read count, add 1, put the result back in count*. The `=` sign means **"store"**, not "is equal to".

✅ **The rule you will follow all week**

Use `const` **by default.** Use `let` **only for values that must change**, such as a counter or the index of the current slide. You will also see `var` in old code: it is the old way, with confusing rules. Don't use it.

> 🧠 **C++ Corner**
>
> `let count = 0;` is like `int count = 0;`, except JavaScript has **no types in declarations**: the same variable could hold a number now and a string later. `const` works like C++ `const` for the box itself.

2. Functions: a recipe with a name

A **function** is a block of code with a name. Writing it does **nothing**; it runs only when it is **called**.

JAVASCRIPT defining and calling a function

```
function increase() {      // define: write the recipe
  count = count + 1;
}

increase();                // call: cook it once
increase();                // cook it again
```

Notice the difference between `increase` and `increase()`:

- `increase()` **with brackets** runs the function **now**.
- `increase` **without brackets** is the function **itself**, like handing someone the recipe card so they can cook it **later**. This is exactly what you give to an event listener.

3. Select → listen → change

Every interactive exercise follows these three steps:

1. **Select:** find the element and keep it in a `const`.

   JAVASCRIPT select

   ```
   const incrementBtn = document.getElementById('increment');
   ```

   You pass the id **without** `#`. You get back the element object, or `null` if no element has that id.

2. **Listen:** tell the browser *"when this happens to this element, run this function"*.

   JAVASCRIPT listen

   ```
   incrementBtn.addEventListener('click', increase);
   ```

   `'click'` is the **event name**. `increase` (no brackets!) is the function the browser will call on every click. A function you hand over to be called later is called a **callback**.

3. **Change:** inside the function, change the page.

   JAVASCRIPT change

   ```
   countEl.textContent = count;
   ```

   `textContent` is the text inside an element. Setting it replaces that text, and the screen updates at once. A number is turned into text automatically.

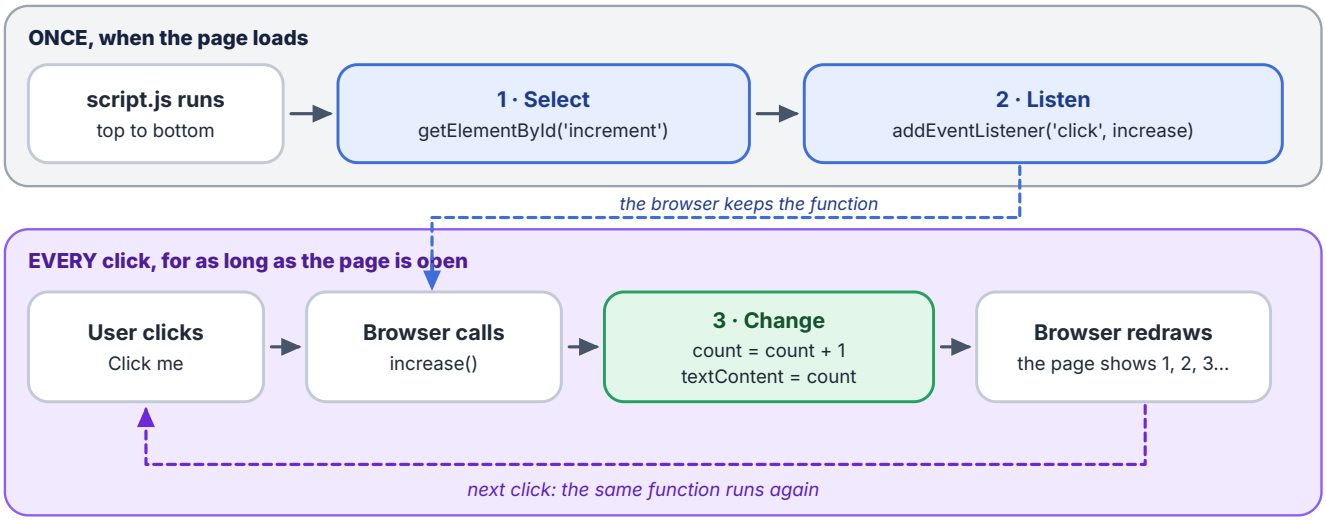


Figure 1.1 *What happens on a click. Your code runs once at page load to **select** and **listen**. After that, the browser calls your function on every click, your function **changes** the DOM, and the browser redraws.*

4. CSS: which rule wins?

Look at these two rules from this exercise:

CSS two rules that both style the Reset button

```css
button:hover {            /* from the starter file */
  background: #6d28d9;
}
.secondary {              /* the rule you will write */
  background: #ffffff;
}
```

When the mouse is over Reset, both rules match and they disagree. The browser picks the **more specific** selector. Count it like a score:

Selector contains	Points
an id (`#count`)	100
a class (`.secondary`) or a pseudo-class (`:hover`)	10 each
an element name (`button`)	1 each

`button:hover` scores 1 + 10 = **11**. `.secondary` scores only **10**, so on hover the purple wins and Reset turns purple. The fix is a rule with a higher score: `.secondary:hover` scores **20**. If two selectors have the **same** score, the one **written later** in the file wins.

Read the starter files

Open the three files in `exercises/03-click-counter/`.

File	What is there	What you will use
`index.html`	complete	`<span id="count">0</span>`, `<button id="increment">`, `<button id="reset" class="secondary">`
`style.css`	almost complete; `button` makes **every** button purple	the empty rule `.secondary { }`
`script.js`	only `TODO` comments	everything

The plan in plain English

1. Select the three elements: the count text and the two buttons.
2. Make a variable `count` that starts at 0. It will change, so use `let`.
3. Write a function `increase` that adds 1 to `count` and shows the new value.
4. Write a function `reset` that sets `count` back to 0 and shows it.
5. Connect them: `increase` runs on a click of **Click me**, `reset` on a click of **Reset**.
6. In CSS, give `.secondary` a white background, grey text and a thin border, plus a hover colour.

✅ Hint 1

You need `document.getElementById(...)` three times, once for each id in the plan. Store each result in a `const` with a clear name, for example `countEl`, `incrementBtn` and `resetBtn`.

✅ Hint 2

The shape of the file is: three selects, one `let`, two functions, two `addEventListener` lines. Each function has two lines inside: one changes `count`, the other copies `count` into `countEl.textContent`.

✅ Hint 3

JAVASCRIPT the inside of increase()

```
count = count + 1;
countEl.textContent = count;
```

And the connection: `incrementBtn.addEventListener('click', increase);` (no brackets after `increase`).

📝 Stop here and build it

Write your code, save, refresh the page and click. Then run `npm test -- 03-click-counter`. Only after that, read on.

The solution, line by line

JAVASCRIPT 03/script.js

```javascript
const countEl = document.getElementById('count');
const incrementBtn = document.getElementById('increment');
const resetBtn = document.getElementById('reset');

let count = 0;

function increase() {
  count = count + 1;
  countEl.textContent = count;
}

function reset() {
  count = 0;
  countEl.textContent = count;
}

incrementBtn.addEventListener('click', increase);
resetBtn.addEventListener('click', reset);
```

Lines	What they do
1–3	**Select.** Find the three elements once, at page load, and keep them in `const`s. They never change, so `const` is right, even though we will change what the elements *show*.
5	The **state** of the app: the number we are counting. `let`, because it changes. It lives **outside** the functions so it remembers its value between clicks.
7–10	`increase`: add 1, then copy the number onto the screen. Changing `count` alone would change nothing visible: the screen only changes when you set `textContent`.
12–15	`reset`: same idea with 0.
17–18	**Listen.** Hand each function to the browser. Nothing runs yet; the browser will call `increase` on every click of `#increment`.

CSS 03/style.css

```css
.secondary {
  background: #ffffff;
  color: #475569;
  border: 1px solid #cbd5e1;
}

.secondary:hover {
  background: #f1f5f9;
}
```

`.secondary` beats the plain `button` rule (10 points against 1), so Reset becomes white with a grey border. Without `.secondary:hover`, the `button:hover` rule (11 points) would turn it purple under the mouse.

🔍 Another way: the repository's `solution/`

The reference solution moves the "copy count onto the screen" line into its own function and writes the listeners as **arrow functions** (you will learn these tomorrow). Both versions pass all four tests.

JAVASCRIPT exercises/03-click-counter/solution/script.js (part)

```
function render() {
  countEl.textContent = String(count);
}

incrementBtn.addEventListener('click', () => {
  count += 1;
  render();
});
```

`count += 1` is a short way of writing `count = count + 1`.

Mistakes beginners make

⚠️ Brackets after the function name

`incrementBtn.addEventListener('click', increase());` **calls** `increase` once, immediately, while the page loads, and hands the browser its result (`undefined`) instead of the function. The count shows 1 and clicks do nothing. Remove the brackets.

⚠️ `let count = 0` inside the function

JAVASCRIPT wrong

```
function increase() {
  let count = 0;           // a NEW box, created on every click
  count = count + 1;
  countEl.textContent = count;
}
```

Every click creates a fresh `count` that starts at 0, so the page shows 1 forever. The state must live **outside** the function.

⚠️ A `#` inside `getElementById`

`document.getElementById('#count')` returns `null` because no element has the id `#count`. The `#` belongs in CSS selectors, not in `getElementById`.

Interview questions

Q1 What is the DOM?

The **Document Object Model**: the tree of objects the browser builds from the HTML. JavaScript reads and changes this tree, and the browser redraws the page to match. The HTML file itself never changes.

Q2 What is the difference between `let`, `const` and `var`?

`const` can't be reassigned; `let` can. Both exist only inside the `{ }` block where they are declared. `var` is the old keyword: it belongs to the whole function, not the block, and can be used before its line (as `undefined`), which causes bugs. Use `const` by default and `let` when the value changes.

Q3 What does `addEventListener` do?

It registers a function (a **callback**) that the browser calls whenever the named event happens on that element. You can add several listeners for the same event, which you can't do with the older `onclick = ...` property (a second assignment replaces the first).

Q4 What is CSS specificity?

The rule that decides which CSS rule wins when several match the same element: ids beat classes, classes and pseudo-classes beat element names, and when the scores are equal the later rule wins.

Practice twist

Add a **−1** button. In `index.html`, add `<button id="decrement" type="button">-1</button>` next to the others; in `script.js`, select it, write `decrease()`, and connect it. Then make sure the count can **never go below 0**. (You'll need `if (count > 0) { ... }`, which you'll meet properly tomorrow. Try it anyway!)

DAY 2 Exercise 01 Fix the Form · Exercise 08 Show / Hide Password

JS

Forms and Toggles

TODAY YOU WILL LEARN

- **HTML forms done properly:** labels, input types, `required`, `fieldset`
- The **submit** event, the **event object** and `preventDefault()`
- Reading what the user typed: `.value` and `.trim()`
- **Template literals** and **arrow functions**
- Decisions with `if` / `else` and `===`
- Changing an element's **properties** and **attributes**

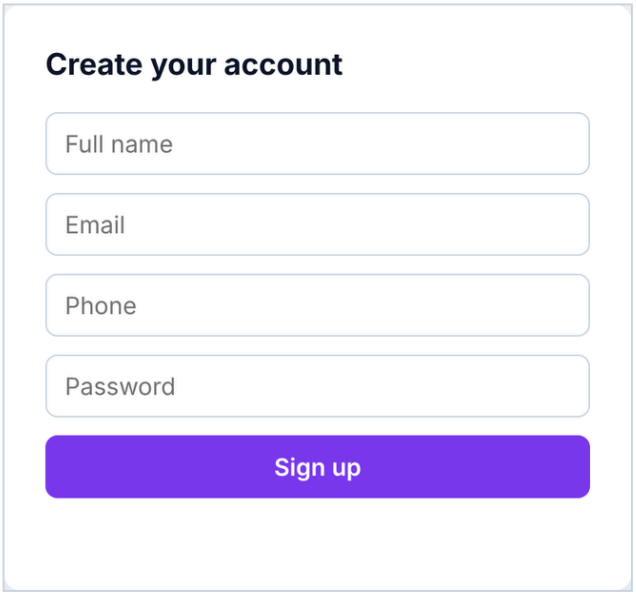
💻 **Today's plan (2 hours)**

Exercise 01 (70 min): new ideas 20 · build 35 · grade and review 15. **Exercise 08** (50 min): new ideas 10 · build 25 · grade and review 15.

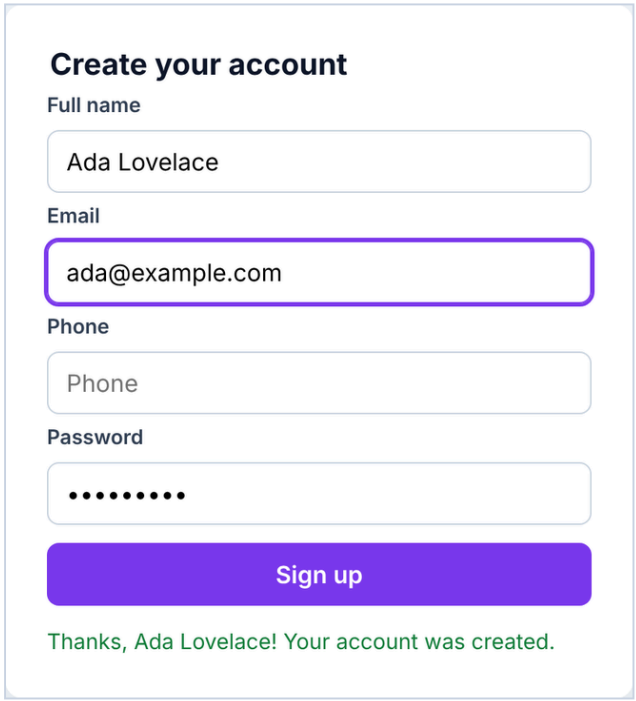
Exercise 01 · Fix the Form

What you are building

START · the starter files



GOAL · after your code



The form **looks** fine, but its HTML is poor: no labels, every field is plain text (even the password is visible), nothing is required. You fix the HTML, add a focus style in CSS, and make the button show a thank-you message. The grader checks:

Test	What it checks
every field has an associated, non-empty label	a `<label for="fullname">` (and email, phone, password) exists and has text
inputs use the correct types	`#email` is `type="email"`, `#phone` is `type="tel"`, `#password` is `type="password"`
required fields are marked required	`#fullname`, `#email`, `#password` have the `required` attribute
fields are grouped in a fieldset with a legend	exactly one `<fieldset>` inside the form, with a `<legend>` directly inside it
submitting shows a message containing the entered name	fills the form, clicks **Sign up**, expects `#result` to contain `Ada Lovelace`

New ideas

1. Labels: the text that belongs to an input

A **placeholder** (the grey text inside an empty box) disappears as soon as you type, and screen readers used by blind users may not read it. A **label** stays visible and is officially connected to its input:

HTML a label connected to an input

```
<label for="email">Email</label>
<input type="email" id="email" />
```

The label's `for` must be **exactly** the input's `id`. Bonus: clicking the label puts the cursor in the input.

2. Input types do real work

Type	What the browser does for you
`type="text"`	nothing special
`type="email"`	checks there's an `@` before submitting; phones show a keyboard with `@`
`type="tel"`	phones show a number pad
`type="password"`	hides the characters as dots

3. `required` and `fieldset`

`required` is a **boolean attribute**: it is either there or not, so you write just the word, `<input ... required />`. The browser then **refuses to submit** while that field is empty and shows a small message.

`<fieldset>` draws a group around related fields, and its first child `<legend>` names the group. Screen readers announce the legend when the user enters the group.

4. The submit event and `preventDefault()`

A `<form>` was invented to **send data to a server**. When it is submitted, the browser sends the data and **loads a new page**. That reload wipes out anything your script showed. Since we have no server, we listen for the form's `submit` event and cancel the reload:

JAVASCRIPT listen to the form, not the button

```
form.addEventListener('submit', (event) => {
  event.preventDefault();      // stop the page from reloading
  // ... now do our own thing
});
```

- Listen to `submit` **on the form**, not `click` on the button. `submit` fires however the form is sent (a click on the button **or** Enter in a field), and only **after** the browser's own checks, such as `required`, have passed.
- The browser passes your function an **event object** (here named `event`) with details about what happened. `event.preventDefault()` means *"don't do your default action"*.

5. Arrow functions

`(event) => { ... }` is a function without a name, written in place. These two do the same thing:

JAVASCRIPT the same listener, two ways

```
function handleSubmit(event) {
  event.preventDefault();
}
form.addEventListener('submit', handleSubmit);

form.addEventListener('submit', (event) => {
  event.preventDefault();
});
```

The arrow form is shorter and keeps the code next to the element it belongs to. From now on, most listeners in this guide use it.

6. Reading the input and building a message

JAVASCRIPT trim and template literals

```
const raw = '   Ada Lovelace  ';
console.log('[' + raw + ']');
console.log('[' + raw.trim() + ']');

const name = raw.trim();
console.log(`Thanks, ${name}! Your account was created.`);
```

OUTPUT

```
[   Ada Lovelace  ]
[Ada Lovelace]
Thanks, Ada Lovelace! Your account was created.
```

- `input.value` is the text currently in an input (always a string).
- `.trim()` removes spaces from **both ends**.
- Text in **backticks** `` ` `` is a **template literal**: `${...}` inside it is replaced by the value. It is much easier to read than `'Thanks, ' + name + '!'`.

7. CSS: `:focus`

`input:focus` styles an input **while the cursor is in it**. Keyboard users rely on it to see where they are. `outline` draws a line outside the border that doesn't change the element's size.

Read the starter files

File	What is there	What you will change
`index.html`	a form `#signup` with four `<input>`s (all `type="text"`), a button, and `<p id="result">`	add labels, fix types, add `required`, wrap in `fieldset` + `legend`
`style.css`	styles for `label`, `input`, `fieldset`, `button` are ready	add an `input:focus` rule
`script.js`	only comments	the submit handler

The plan in plain English

1. **HTML:** put a `<fieldset>` around the inputs; its first child is a `<legend>`. Before each input, add a `<label for="...">` with the input's id. Fix the three types. Add `required` to name, email and password.
2. **CSS:** write `input:focus` with a coloured outline.
3. **JS:** select the form, the name input and `#result`. On `submit`: prevent the reload, read and trim the name, write the message into `#result`.

✅ Hint 1

The `<h1>Create your account</h1>` can become the `<legend>`: the legend **is** the heading of the group. Keep the button and `#result` after the `</fieldset>`.

✅ Hint 2

The JavaScript is 3 selects plus one listener with 3 lines inside. The listener goes on the **form** (`'submit'`), not on the button.

✅ Hint 3

JAVASCRIPT the inside of the submit listener

```
event.preventDefault();
const name = nameInput.value.trim();
result.textContent = `Thanks, ${name}! Your account was created.`;
```

> 📝 **Stop here and build it**
>
> Then run `npm test -- 01-fix-the-form`.

The solution, line by line

HTML 01/index.html (the form)

```
<form id="signup" class="card">
  <fieldset>
    <legend>Create your account</legend>

    <label for="fullname">Full name</label>
    <input type="text" id="fullname" placeholder="Full name" required />

    <label for="email">Email</label>
    <input type="email" id="email" placeholder="Email" required />

    <label for="phone">Phone</label>
    <input type="tel" id="phone" placeholder="Phone" />

    <label for="password">Password</label>
    <input type="password" id="password" placeholder="Password" required />
  </fieldset>

  <button type="submit">Sign up</button>
  <p id="result" class="result" aria-live="polite"></p>
</form>
```

- The heading moved into `<legend>`, so the form still has a title and the grader finds the legend **directly inside** the fieldset.
- Every `for` matches an `id` exactly. The phone is not required (the task lists only name, email and password).
- `aria-live="polite"` on `#result` was already there: it tells screen readers to read the message aloud when it appears.

CSS 01/style.css (the new rule, where the TODO was)

```
input:focus {
  outline: 2px solid #7c3aed;
  border-color: #7c3aed;
}
```

CSS 01/style.css (two lines added to the existing fieldset rule)

```
fieldset {
  border: 0;
  padding: 0;
  margin: 0;
  display: grid;
  gap: 8px;
}
```

The `fieldset` rule already existed; adding `display: grid` and `gap: 8px` spaces the labels and inputs evenly. (You'll learn grid on Day 3. For now: *"stack the children with an 8px gap"*.) The grader doesn't check it, but without it the labels sit right on top of the inputs above them.

JAVASCRIPT 01/script.js

```
1  const form = document.getElementById('signup');
2  const nameInput = document.getElementById('fullname');
3  const result = document.getElementById('result');
4
5  form.addEventListener('submit', (event) => {
6    event.preventDefault();
7    const name = nameInput.value.trim();
8    result.textContent = `Thanks, ${name}! Your account was created.`;
9  });
```

Lines	What they do
1–3	**Select** the form, the name box and the message paragraph.
5	**Listen** for `submit` on the form. The browser hands us the event object.
6	Cancel the page reload, or the message would vanish a moment after it appears.
7	**Read** what was typed, without spaces at the ends.
8	**Change** the page: put the message in `#result`.

Mistakes beginners make

⚠️ `required="true"`

It looks logical but the grader **fails** it (we tried: *Expected "" · Received "true"*). A boolean attribute is written as the bare word: `required`. Even `required="false"` would still make the field required, because only the **presence** of the attribute matters.

⚠️ **Forgetting** `event.preventDefault()`

The message appears for a split second, then the page reloads and it's gone. We removed that line and ran the grader: the last test failed with *Received string: ""*.

⚠️ **Listening for** `click` **on the button**

It seems to work, but the click happens **before** the browser checks the `required` fields. Your code runs even when the form is half empty, and if you call `preventDefault()` on the click, the browser's check never runs at all. A `submit` listener runs only once the checks have passed.

Interview questions

Q1 Why use a `<label>` instead of a placeholder?

A label is always visible, is read by screen readers, and clicking it focuses its input. A placeholder vanishes when you type and is not a reliable name for the field.

Q2 What does `event.preventDefault()` do? Give two uses.

It cancels the browser's default action for that event. Stopping a form submit from reloading the page, and stopping a link (`<a>`) from navigating, are the two classic uses.

Q3 Why `type="email"` and not `type="text"` ?

The browser checks the format before submitting, and mobile phones show a keyboard with `@` . It's free validation and better usability.

Q4 What is semantic HTML?

Using the element that **means** the right thing: `<button>` for buttons, `<label>` for labels, `<nav>` , `<main>` , `<fieldset>` . It helps screen readers, search engines and other developers, at no cost.

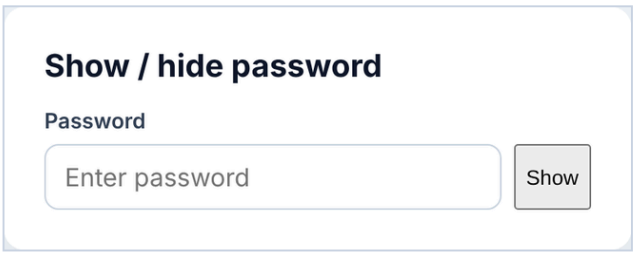
Practice twist

Make the message friendlier: if the name is empty after trimming (someone typed only spaces), show *"Please enter your name."* instead. Use `if (name === '') { ... } else { ... }` . You'll see `if` properly in the next exercise.

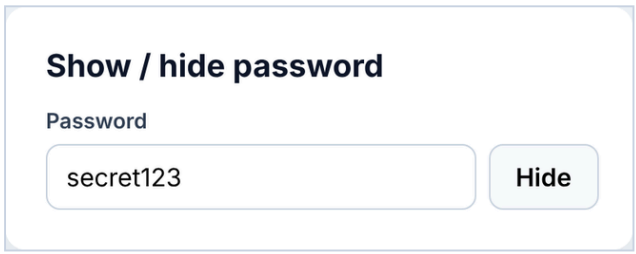
Exercise 08 · Show / Hide Password

What you are building

START · the starter files



GOAL · after your code



A **Show** button next to a password field. Clicking it reveals the password and the button says **Hide**; clicking again hides it. The grader checks:

Test	What it checks
starts masked with a “Show” button	`#password` has `type="password"`, the button says *Show*
reveals the password on click and updates the label	after one click: `type="text"` and *Hide*
hides the password again on the second click	after two clicks: back to `password` and *Show*
keeps aria-pressed in sync	after one click `aria-pressed="true"`, after two `"false"`

New ideas

1. Decisions: `if` / `else`

JAVASCRIPT if / else

```
if (passwordInput.type === 'password') {
  // runs when the condition is true
} else {
  // runs otherwise
}
```

Compare with **three** equals signs, `===`. Two (`==`) converts types before comparing, which gives surprises:

JAVASCRIPT == versus ===

```
console.log(5 == '5', 5 === '5');
console.log('password' === 'password', 'password' === 'text');
```

OUTPUT

```
true false
true false
```

`5 == '5'` is `true` because `==` turns the text `'5'` into a number first. `===` says the number 5 and the text `'5'` are different, which is what you almost always want. **Always use** `===` (and `!==` for “not equal”).

2. Element properties: read them and change them

An element object has **properties** you can read and change with a dot. For an input, `input.type` is its type:

JAVASCRIPT type into the console of exercise 08

```
const input = document.getElementById('password');
input.type             // 'password'
input.type = 'text'    // the dots turn into letters
```

3. Attributes with `setAttribute`

Some information lives only in **attributes**, such as `aria-pressed`. **ARIA** attributes describe the page to screen readers: `aria-pressed="true"` says *"this toggle button is switched on"*. Set any attribute with:

JAVASCRIPT setAttribute

```
toggleBtn.setAttribute('aria-pressed', 'true');
```

Attribute values are always **strings**, so write `'true'` with quotes.

Read the starter files

File	What is there	What you will use
`index.html`	complete	`<input id="password" type="password">`, `<button id="toggle-password" aria-pressed="false">Show</button>`
`style.css`	almost complete	the empty `#toggle-password { }` rule
`script.js`	only comments	everything

The plan in plain English

1. Select the input and the button.
2. On a click of the button, **ask**: is the password hidden right now (type is `'password'`)?
3. **If yes:** show it (type `'text'`), set the label to *Hide*, set `aria-pressed` to `'true'`.
4. **Otherwise:** hide it (type `'password'`), label *Show*, `aria-pressed` `'false'`.
5. CSS: padding, rounded corners, a light background and `cursor: pointer`.

✅ **Hint 1**

The current state is stored in the page itself: `passwordInput.type` **is** the answer to "hidden or not?". You don't need a separate variable.

✅ Hint 2

Inside the listener: one `if (...) { 3 lines } else { 3 lines }`. The three lines change `type`, `textContent` and `aria-pressed`.

✅ Hint 3

JAVASCRIPT the first branch

```
passwordInput.type = 'text';
toggleBtn.textContent = 'Hide';
toggleBtn.setAttribute('aria-pressed', 'true');
```

📝 Stop here and build it

Then run `npm test -- 08-show-hide-password`.

The solution, line by line

JAVASCRIPT 08/script.js

```
1  const passwordInput = document.getElementById('password');
2  const toggleBtn = document.getElementById('toggle-password');
3
4  toggleBtn.addEventListener('click', () => {
5    if (passwordInput.type === 'password') {
6      passwordInput.type = 'text';
7      toggleBtn.textContent = 'Hide';
8      toggleBtn.setAttribute('aria-pressed', 'true');
9    } else {
10     passwordInput.type = 'password';
11     toggleBtn.textContent = 'Show';
12     toggleBtn.setAttribute('aria-pressed', 'false');
13   }
14 });
```

Lines	What they do
1–2	Select the input and the button.
4	Listen for clicks. The arrow function has no parameter because we don't need the event object.
5	Read the current state: hidden or visible?
6–8	It was hidden: reveal it, and make the button and its ARIA state say so.
10–12	It was visible: hide it again.

CSS 08/style.css

```css
#toggle-password {
  padding: 10px 16px;
  border: 1px solid #cbd5e1;
  border-radius: 9px;
  background: #f8fafc;
  font: inherit;
  font-weight: 600;
  cursor: pointer;
}
```

`font: inherit` matters: **buttons and inputs don't use the page's font by default**. Without it, the button text appears in a small system font.

🔍 **Another way: the ternary operator**

`condition ? a : b` means *"if condition is true, the value is* `a` *, otherwise* `b` *"*. The repository's `solution/` uses it to shrink the listener:

JAVASCRIPT exercises/08-show-hide-password/solution/script.js (part)

```javascript
toggle.addEventListener('click', () => {
  const isMasked = input.type === 'password';
  input.type = isMasked ? 'text' : 'password';
  toggle.textContent = isMasked ? 'Hide' : 'Show';
  toggle.setAttribute('aria-pressed', String(isMasked));
});
```

`isMasked` is `true` or `false` (a **boolean**). `String(isMasked)` turns it into `'true'` or `'false'`. Use whichever version you can explain better; in an interview, clear beats short.

Mistakes beginners make

⚠️ **One `=` inside the `if`**

`if (passwordInput.type = 'password')` **sets** the type instead of comparing it, and the condition is always true. One `=` stores, three `===` compare.

⚠️ **Comparing with the wrong case**

`'Password' === 'password'` is `false`. Strings must match exactly, letter for letter.

Interview questions

Q5 What is the difference between `==` and `===`?

`===` (strict equality) compares value **and** type without converting anything. `==` converts types first, so `5 == '5'` and `0 == ''` are both `true`. Always use `===` unless you have a specific reason.

Q6 What is the difference between a property and an attribute?

Attributes are written in the HTML (`aria-pressed="false"`) and are always strings. Properties live on the element object in JavaScript (`input.type` , `input.value`). Many attributes have a matching property; `input.value` holds what the user typed **now**, while the `value` attribute is only the starting value.

Q7 What are ARIA attributes for?

They give screen readers information that plain HTML doesn't: whether a toggle is pressed (`aria-pressed`), whether a section is expanded (`aria-expanded`), and which tab is selected (`aria-selected`). Graders and interviewers like to see them kept in sync.

Practice twist

Make the button show an eye icon instead of text: `'👁'` for Show and `'🙈'` for Hide. Then add a second password field, **Confirm password**, with its own toggle. Can you write one function that works for both buttons?

DAY 3 Exercise 02 Profile Card · Exercise 04 Center It

JS

CSS Day: Boxes, Rows and Centering

TODAY YOU WILL LEARN

- The **box model**: content, padding, border, margin
- Units and colours: `px`, `rem`, `%`, `#hex`, `rgba()`
- Rounded corners, shadows and text styles
- **Flexbox**: putting things in a row and lining them up
- Three ways to **center** anything: flexbox, grid and absolute positioning

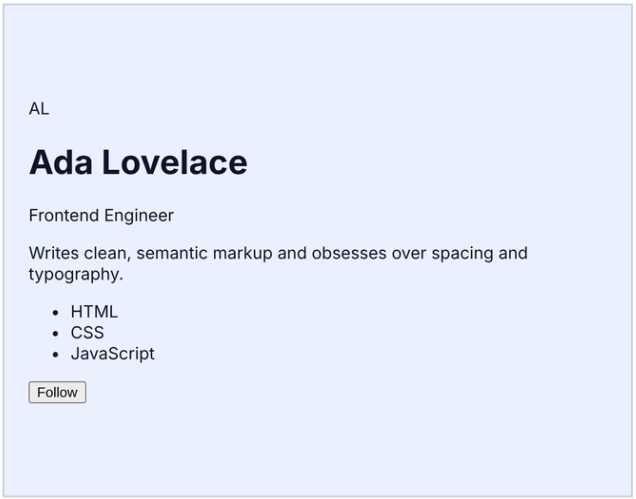
💻 **Today's plan (2 hours)**

Both exercises are **CSS only**: you don't touch the HTML or JavaScript. **Exercise 02** (70 min) · **Exercise 04** (50 min). Keep the browser open next to the editor and refresh after every few lines: CSS is learned by watching it change.

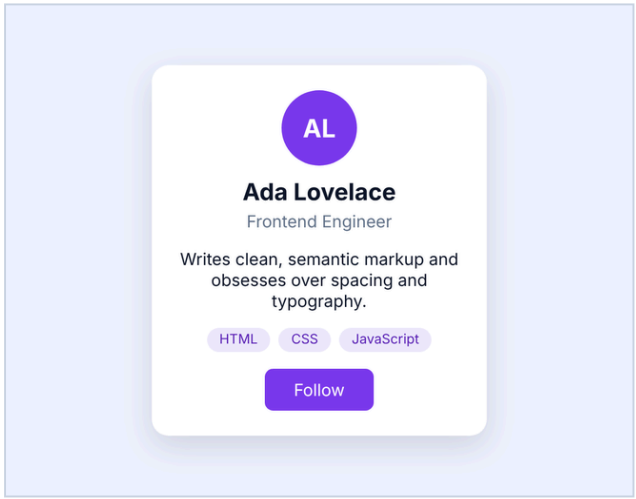
Exercise 02 · Style the Profile Card

What you are building

START · the starter files



GOAL · after your code



The HTML is complete but unstyled. You write seven CSS rules. The grader measures the result with `getComputedStyle`, the browser's final answer for each property:

Test	What it checks
card has rounded corners, padding and a shadow	`.profile-card`: radius ≥ 12px, padding ≥ 16px, a `box-shadow`, a background colour
avatar is a circle	`.avatar`: width ≥ 64px, width = height, radius at least half the width
name is bold and role is muted	`.name` weight ≥ 600; `.role` colour differs from the normal text colour
tags render as styled chips in a row	`.tags`: no bullets, `display: flex`; `.tag`: padding, background, rounded corners
follow button looks clickable	`.follow-btn`: `cursor: pointer`, a background, **white** text, padding

New ideas

1. The box model

Every element is a rectangle made of four layers:

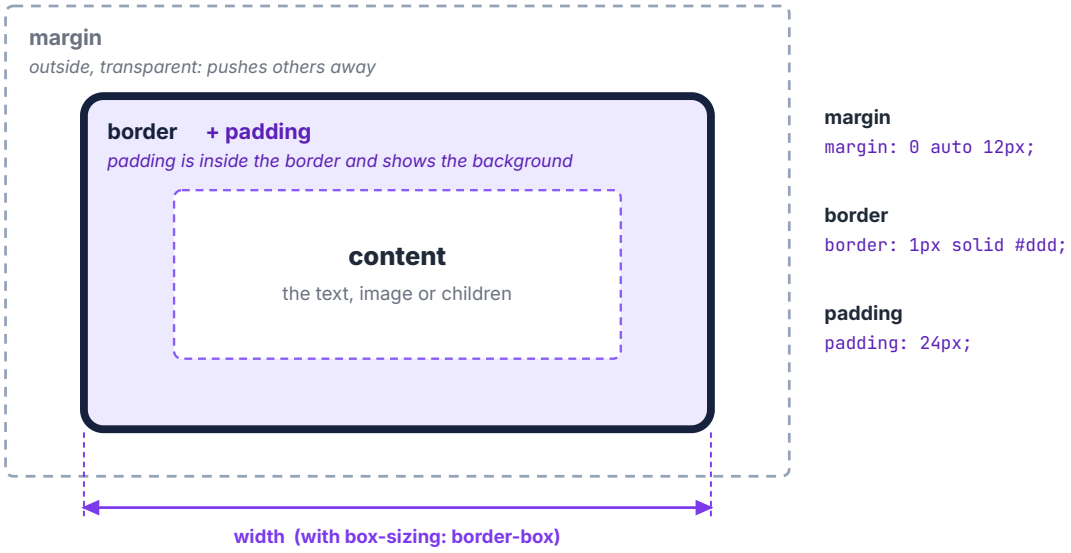


Figure 3.1 *From the inside out: the **content**, the **padding** (space inside the border, which shows the background colour), the **border**, and the **margin** (space outside, always transparent, that pushes other elements away).*

CSS the four layers in CSS

```css
.card {
  width: 320px;           /* content width                        */
  padding: 24px;          /* inside space, on all four sides      */
  border: 1px solid #ddd; /* thickness, style, colour             */
  margin: 0 auto 12px;    /* top 0, left & right auto, bottom 12px */
}
```

Every starter file begins with `* { box-sizing: border-box; }`. It makes `width` include the padding and border, so a `320px` card is really 320px wide. Without it, padding is **added** to the width, which surprises everyone.

Shorthand with several values goes clockwise from the top: `margin: 4px 0 12px` means top 4, left and right 0, bottom 12. One value means all four sides; two values mean *top-and-bottom* then *left-and-right*.

2. Units and colours

Unit	Means	Use it for
`px`	screen pixels	borders, small fixed sizes, shadows
`rem`	times the page's base font size (usually 16px): `1.5rem` = 24px	font sizes
`%`	percent of something else: `border-radius: 50%` = half of the element's size	circles, widths

Colours: `#7c3aed` (red, green, blue in hex), `white`, or `rgba(0, 0, 0, 0.12)`, which is black at 12% opacity, perfect for soft shadows.

3. Corners, shadows and text

Property	Example	Effect
`border-radius`	`16px` · `50%` · `999px`	rounded corners · a circle (if width = height) · a pill
`box-shadow`	`0 10px 30px rgba(0,0,0,0.12)`	x offset, y offset, blur, colour: a soft shadow below
`font-size`	`1.4rem`	text size
`font-weight`	`400` normal · `600` semi-bold · `700` bold	thickness
`color`	`#64748b`	**text** colour (`background` is the fill)
`text-align`	`center`	lines up the text inside the box

4. Flexbox: a row of items

`display: flex` on a parent lines up its **children** in a row:

CSS the four flexbox properties you need

```css
.tags {
  display: flex;            /* children side by side            */
  gap: 8px;                 /* space between them               */
  justify-content: center;  /* along the row: start | center | space-between */
  align-items: center;      /* across the row: vertical alignment */
}
```

A `<ul>` has bullets and a left padding by default: remove them with `list-style: none` and `padding: 0`.

Flexbox also centers **one** child: on `.avatar`, `display: flex` + `align-items: center` + `justify-content: center` puts the letters "AL" in the middle of the circle.

5. Centering a block horizontally

A block with a fixed width is centered by `margin: 0 auto`. The browser shares the leftover space equally between the left and right margins.

Read the starter files

The HTML is an `<article class="profile-card">` with these children, in order: `.avatar` ("AL"), `h1.name`, `p.role`, `p.bio`, `ul.tags` with three `li.tag`, and `button.follow-btn`. The CSS file has **seven empty rules** with a `TODO` above each. Fill them in; don't change the HTML.

The plan in plain English

Go rule by rule, refreshing the browser after each:

1. **Card:** a width (and `max-width: 100%` for small screens), white background, radius 16px, padding 24px, a soft shadow, centered text.
2. **Avatar:** 72 × 72, `border-radius: 50%`, purple background, white bold text, flexbox to center the letters, `margin: 0 auto 12px` to center the circle itself.
3. **Name:** bigger and bold.
4. **Role:** a grey colour.
5. **Tags:** remove bullets and padding, make it a centered flex row with a gap. **Tag:** small padding, light purple background, pill corners, smaller text.
6. **Button:** padding, no border, rounded, purple background, white text, `font: inherit`, `cursor: pointer`.

> ✅ **Hint 1**
>
> If the avatar is an oval, its width and height differ. If it's a rounded square, the radius is less than 50%.

> ✅ **Hint 2**
>
> The tags still show dots? You set `list-style` on `.tag` instead of `.tags`. The bullets belong to the list, the `<ul>`.

> ✅ **Hint 3**
>
> The grader wants the button text **exactly white**: `color: #ffffff` (or `#fff`, or `white`) all become `rgb(255, 255, 255)`.

> 📝 **Stop here and build it**
>
> Then run `npm test -- 02-style-the-profile-card`.

The solution

CSS 02/style.css

```css
.profile-card {
  width: 320px;
  max-width: 100%;
  background: #ffffff;
  border-radius: 16px;
  padding: 24px;
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.12);
  text-align: center;
}

.avatar {
  width: 72px;
  height: 72px;
  border-radius: 50%;
  background: #7c3aed;
  color: #ffffff;
  font-size: 1.5rem;
  font-weight: 700;
  display: flex;
  align-items: center;
  justify-content: center;
  margin: 0 auto 12px;
}

.name {
  font-size: 1.4rem;
  font-weight: 700;
  margin: 0;
}

.role {
  color: #64748b;
  margin: 4px 0 12px;
}

.tags {
  list-style: none;
  padding: 0;
  display: flex;
  justify-content: center;
  gap: 8px;
}

.tag {
  padding: 4px 12px;
  background: #ede9fe;
```

```
  color: #5b21b6;
  border-radius: 999px;
  font-size: 0.8rem;
}

.follow-btn {
  padding: 10px 28px;
  border: none;
  border-radius: 8px;
  background: #7c3aed;
  color: #ffffff;
  font: inherit;
  cursor: pointer;
}
```

A few choices explained:

- `max-width: 100%` lets the 320px card shrink on a phone narrower than 320px.
- `.avatar` **uses flexbox twice over:** its own `display: flex` centers the letters inside it, while `margin: 0 auto` centers the circle inside the card. (`text-align: center` on the card centers **text**, not a block with its own width.)
- `margin: 0` **on** `.name` removes the large default margin every `<h1>` has.
- `border-radius: 999px` is a trick for a pill shape: any radius bigger than half the height gives perfectly round ends.
- `font: inherit` on the button makes it use the page font, as you saw on Day 2.

Mistakes beginners make

⚠️ `color` **when you meant** `background`

`color` is the **text** colour. A purple button needs `background: #7c3aed` **and** `color: #ffffff`.

⚠️ **Forgetting the semicolon**

CSS one missing ; breaks the next line too

```
.name {
  font-size: 1.4rem
  font-weight: 700;
}
```

The browser reads `1.4rem font-weight: 700` as one broken value and ignores **both** lines. If a property “does nothing”, check the line above it.

Interview questions

Q1 Explain the CSS box model.

Every element is a box: content, then padding, then border, then margin. Padding is inside the border and shows the background; margin is outside and transparent. `box-sizing: border-box` makes `width` include padding and border, which is easier to reason about.

Q2 What is the difference between `rem` and `px`?

`px` is a fixed size. `rem` is relative to the root font size (16px by default), so text in `rem` grows when a user increases their browser's font size. That's better for accessibility.

Q3 How do you make a circle in CSS?

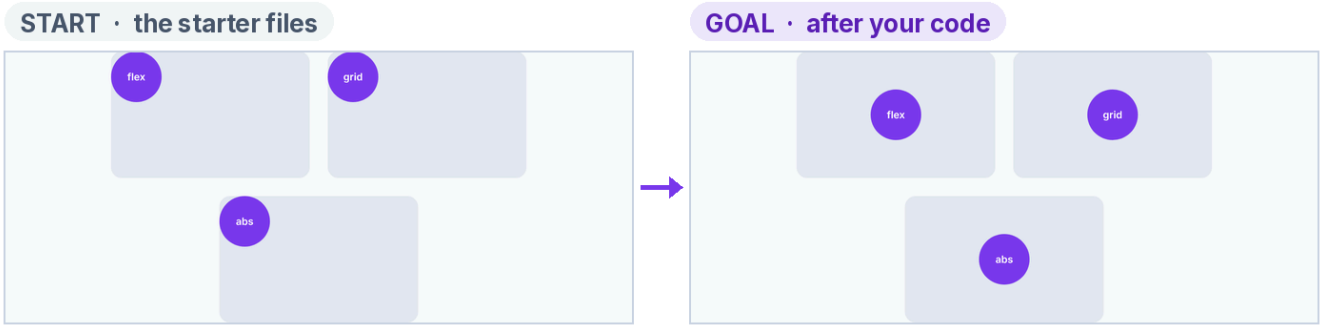
Equal width and height plus `border-radius: 50%`.

Practice twist

Add a `.follow-btn:hover` rule with a darker purple, and a `transition: background 0.2s;` on `.follow-btn` so the colour fades instead of jumping. Then give the card a hover effect: `transform: translateY(-4px)`.

Exercise 04 · Center It

What you are building



Three grey boxes (`.stage`), each with a purple dot. Center each dot **horizontally and vertically**, using a different technique each time. *"How do you center a div?"* is one of the most asked CSS interview questions, and after today you'll have three answers. The grader checks that `#stage-flex` uses `display: flex` , `#stage-grid` uses `display: grid` , the third dot has `position: absolute` , and in all three the dot's center is within 2 pixels of the box's center.

New ideas

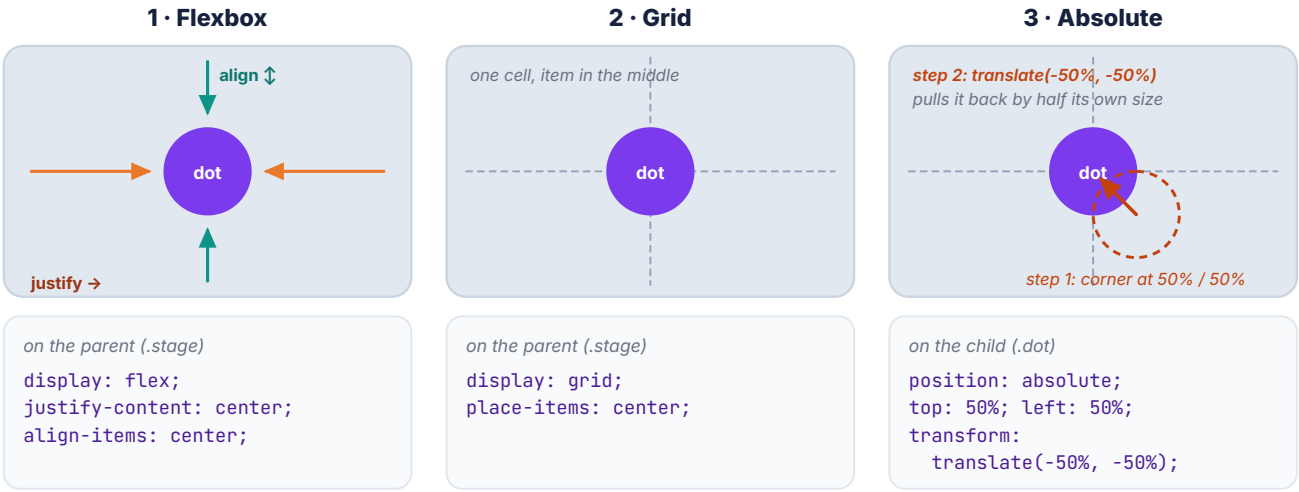


Figure 3.2 *Three ways to center. **Flexbox:** one property per direction. **Grid:** one property for both. **Absolute:** move the dot's top-left corner to the middle, then pull it back by half its own size.*

1. Flexbox has two axes

In a flex row, `justify-content` works **along** the row (left ↔ right) and `align-items` works **across** it (top ↕ bottom). Center on both and the child sits in the middle:

CSS centering with flexbox

```css
.box {
  display: flex;
  justify-content: center;  /* horizontal */
  align-items: center;      /* vertical   */
}
```

2. Grid centers in one line

`display: grid` turns the box into a grid; with one child, it's a single cell. `place-items: center` centers the child in both directions at once.

3. Positioning

`position`	What it does
`static`	the default: the element sits in the normal flow
`relative`	stays in place, but becomes the **reference box** for absolute children
`absolute`	taken out of the flow and placed with `top` / `left` / `right` / `bottom`, measured from the nearest **positioned** ancestor (`relative` , `absolute` or `fixed`)

`top: 50%; left: 50%` moves the dot's **top-left corner** to the middle of the stage, so the dot hangs down and to the right. `transform: translate(-50%, -50%)` then moves it up and left by **half of its own** width and height. In `translate`, percentages refer to the element **itself**, not its parent: that is the whole trick.

Read the starter files

`.stage` already has `position: relative` (so it's the reference box), a fixed size, and a grey background. `.dot` is a 56px purple circle. Three rules are empty: `#stage-flex`, `#stage-grid`, and `#stage-abs .dot`. Notice the third one targets the **dot**, while the other two target the **stage**.

The plan in plain English

1. `#stage-flex`: make it a flex container and center on both axes.
2. `#stage-grid`: make it a grid and use `place-items`.
3. `#stage-abs .dot`: position the dot absolutely at 50% / 50%, then translate it back by −50% / −50%.

> ✅ **Hint**
>
> For flexbox and grid, the rule goes on the **parent** (the stage): a parent controls where its children sit. For the absolute method, the rule goes on the **child** (the dot): the child places itself.

> 📝 **Stop here and build it**
>
> Then run `npm test -- 04-center-it`.

The solution

CSS 04/style.css

```css
#stage-flex {
  display: flex;
  justify-content: center;
  align-items: center;
}

#stage-grid {
  display: grid;
  place-items: center;
}

#stage-abs .dot {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
}
```

Which should you use in real work? **Flexbox or grid**, because they keep the element in the normal flow. The absolute method is useful for something that must sit **on top** of other content, such as a badge on a card or a loading spinner over a page.

Mistakes beginners make

⚠️ `align-items` **without** `display: flex`

`justify-content` and `align-items` do nothing unless the same element has `display: flex` (or `grid`). They are instructions **for a flex container**.

⚠️ **Absolute without a positioned parent**

If no ancestor has `position: relative`, an absolute element is placed relative to the **whole page**: the dot flies to the middle of the screen. Here `.stage` already has `position: relative`, but in your own projects you must add it.

Interview questions

Q4 How do you center a div horizontally and vertically?

On the parent, `display: flex; justify-content: center; align-items: center;`, or `display: grid; place-items: center;`. Without flex or grid: `position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%);` with a positioned parent.

Q5 What is the difference between `position: relative` and `absolute`?

`relative` keeps the element in the normal flow (you can nudge it with `top` / `left`) and makes it the reference for absolute children. `absolute` removes the element from the flow and places it relative to the nearest positioned ancestor.

Q6 When would you use grid instead of flexbox?

Flexbox lays things out in **one** direction (a row or a column); grid lays them out in **two** (rows and columns at once). Navbars and button groups suit flexbox; card galleries and page layouts suit grid.

Practice twist

Add a fourth stage to the HTML, `<section class="stage" id="stage-margin">`, and center its dot with **flexbox on the stage plus `margin: auto` on the dot**, without `justify-content` or `align-items`. (Inside a flex container, `margin: auto` soaks up the free space on every side.)

DAY 4 Exercise 05 ⭐ · arrays · filter · creating elements

JS

The Real Question: E-commerce Search Box

TODAY YOU WILL LEARN

- **Arrays**: lists of values, and looping with `forEach`
- `filter`: keeping only the items you want
- Case-insensitive matching with `toLowerCase()` and `includes()`
- The `input` event: reacting to every key press
- **Creating elements** with `createElement` and `appendChild`
- The most useful pattern in this guide: **one `render()` function** that redraws from the data

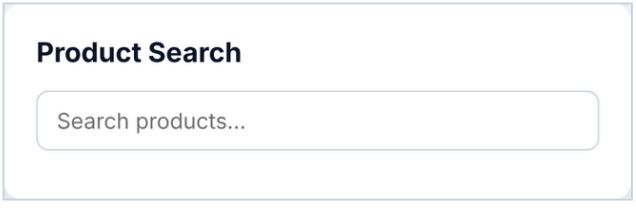
💻 **Today's plan (2 hours)**

This is the question a candidate reported **word for word** from the real round, so today has one exercise and extra time. **0–35** new ideas (type every example in the console) · **35–45** starter files · **45–95** build · **95–105** grade · **105–120** review. Tomorrow morning, redo it from the starter files in under 25 minutes.

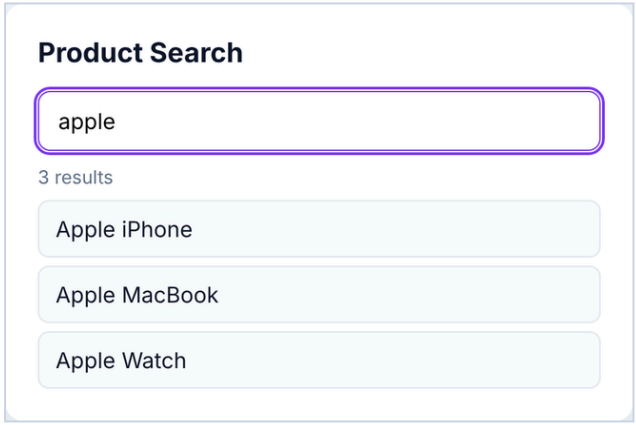
Exercise 05 · E-commerce Search Box ⭐

What you are building

START · the starter files



GOAL · after your code



A list of 8 products under a search input. As you type, only matching products stay, a counter says how many, and *No results found* appears when nothing matches. The grader checks:

Test	What it does	Expects
renders all 8 products before typing	just opens the page	8 `.result-item`s; `#count` contains `8`
filters case-insensitively as the user types	types `APPLE`	3 items; `#count` contains `3`; `#empty` hidden
shows “No results found” for a non-matching query	types `zzzz`	0 items; `#empty` visible
clearing the query restores all results	types `apple`, then clears the box	8 items; `#empty` hidden
a partial query matches substrings	types `phone`	2 items (Apple **iPhone**, Sony Head**phone**s)

New ideas

1. Arrays and `forEach`

An **array** is an ordered list in square brackets. Positions are counted **from 0**, and `.length` is how many items there are. `forEach` runs a function **once for each item**, passing the item in:

JAVASCRIPT an array and forEach

```javascript
const products = ['Apple iPhone', 'Samsung Galaxy', 'Apple Watch'];

products.forEach((product) => {
  console.log('->', product);
});
```

OUTPUT

```
-> Apple iPhone
-> Samsung Galaxy
-> Apple Watch
```

Read it as: *“for each product in products, print it”*. `products[0]` is `'Apple iPhone'`, and `products.length` is 3.

🧠 **C++ Corner**

`products.forEach((product) => { ... })` is the same idea as `for (const auto& product : products) { ... }`. A normal `for (let i = 0; i < products.length; i++)` loop also works in JavaScript.

2. `filter`: keep only what you want

`filter` also calls your function once per item, but your function must **answer true or false**. Items that get `true` are kept in a **new** array; the original is not changed.

JAVASCRIPT filter with a case-insensitive match

```
const query = 'APPLE'.toLowerCase();
const matches = products.filter((product) => {
  return product.toLowerCase().includes(query);
});
console.log(query);
console.log(matches);
console.log(matches.length);
```

OUTPUT

```
apple
[ 'Apple iPhone', 'Apple Watch' ]
2
```

- `toLowerCase()` gives a lower-case copy of a string. Lower-casing **both sides** makes the match ignore case: `'APPLE'`, `'apple'` and `'Apple'` all become `'apple'`.
- `a.includes(b)` is `true` if the text `b` appears anywhere inside `a`. That's why `'phone'` finds *Sony Head**phone**s*.
- `return` sends the answer back to `filter`.

3. A free gift: the empty string

JAVASCRIPT every string contains the empty string

```
console.log('Apple Watch'.includes(''));
console.log(products.filter((p) => p.toLowerCase().includes('')).length);
```

OUTPUT

```
true
3
```

When the search box is empty, the query is `''`, which every product contains, so the filter **keeps everything**. You don't need a special case for "show all when empty".

The second line also shows a **short arrow function**: with no `{ }`, the value after `=>` is returned automatically. `(p) => p.toLowerCase().includes('')` is the same as `(p) => { return p.toLowerCase().includes(''); }`.

4. The `input` event

`'input'` fires on **every change** to a text box: each key press, each delete, a paste. (The `'change'` event only fires when the user **leaves** the box, which is too late for live search.)

JAVASCRIPT react to typing

```
searchInput.addEventListener('input', render);
```

Inside, `searchInput.value` is the full text in the box **at that moment**.

5. Creating elements

To add an item to the page, you **create** an element, **fill** it, and **attach** it to a parent that's already on the page:

JAVASCRIPT create, fill, attach

```
const li = document.createElement('li');   // 1. create (not on the page yet)
li.className = 'result-item';               // 2. fill: give it a class...
li.textContent = 'Apple iPhone';            //    ...and its text
resultsList.appendChild(li);                // 3. attach: now it appears
```

`className` sets the whole `class` attribute. To **empty** a parent, set `resultsList.innerHTML = ''`, which removes everything inside it.

6. Showing and hiding with `hidden`

Every element has a `hidden` property. `el.hidden = true` hides it; `el.hidden = false` shows it. `#empty` starts with the `hidden` attribute in the HTML, so it's invisible at first.

`matches.length > 0` is itself `true` or `false`, so this one line hides the message when there are matches and shows it when there are none:

JAVASCRIPT a comparison is already a true/false value

```
emptyText.hidden = matches.length > 0;
```

7. The render pattern

Here is the idea that makes this exercise (and Days 5 and 9) easy:

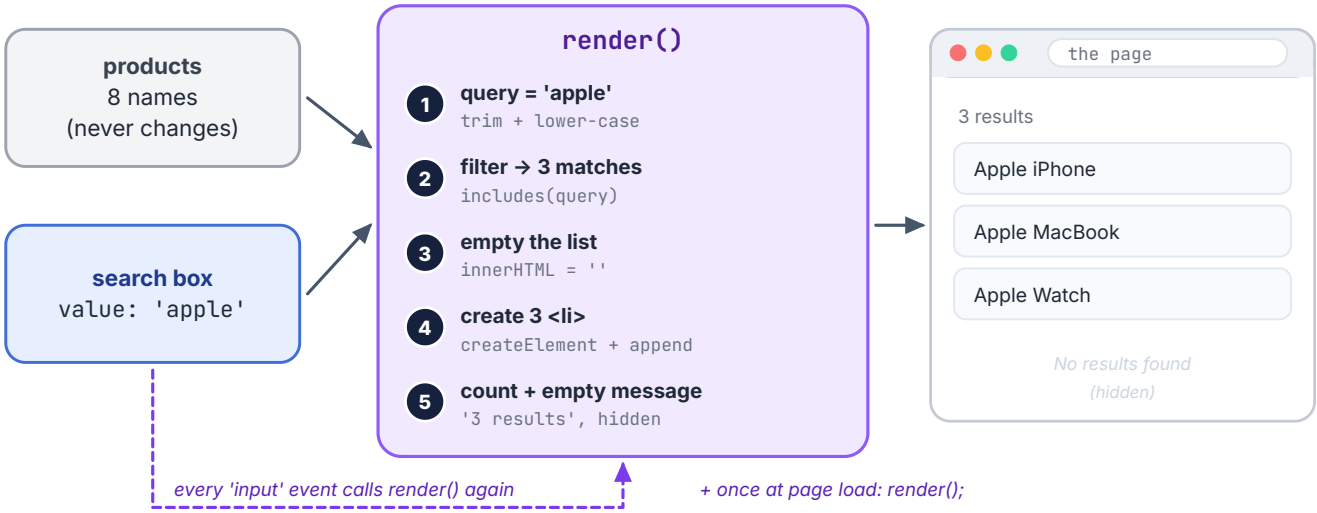


Figure 4.1 *One `render()` function rebuilds the list **from the data** every time. It doesn't try to work out which items to add or remove: it throws the old list away and draws the new one.*

1. **Data:** the `products` array never changes.
2. **Input:** whatever is in the search box now.
3. `render()`: compute the matches, empty the list, create one `<li>` per match, update the count and the empty message.
4. Call `render()` **once at the start** (so 8 products show before any typing) and **again on every** `input`.

Read the starter files

File	What is there	What you will use
`index.html`	complete	`#search` (input), `#count` (p), `#results` (empty ul), `#empty` (p, `hidden`)
`style.css`	done except two rules	`.result-item` and `.empty`
`script.js`	the `products` array (**don't change it**) and `TODO` comments	everything below the array

The plan in plain English

1. Select `#search`, `#results`, `#count` and `#empty`.
2. Write `render()`:
    1. Read the box, trim it, lower-case it: that's the **query**.
    2. `filter` the products: keep those whose lower-cased name `includes` the query.
    3. Empty `#results`.
    4. For each match: create an `<li class="result-item">` with the name and append it.

5. Set `#count` to, for example, `3 results`.
6. Hide `#empty` when there are matches; show it when there are none.

3. Listen for `input` on the search box and call `render`.
4. Call `render()` once at the end of the file.
5. CSS: give `.result-item` padding, a light background, rounded corners and a hover colour; make `.empty` centered and grey.

✅ Hint 1

Write `render()` so that it only does step 2.4 first, with `products` instead of `matches`. When you see all 8 items on load, add the filter.

✅ Hint 2

The skeleton:

JAVASCRIPT render() skeleton

```
function render() {
  const query = /* box text, trimmed, lower-cased */;
  const matches = products.filter((product) => /* true or false */);
  resultsList.innerHTML = '';
  matches.forEach((product) => { /* create, fill, attach */ });
  countText.textContent = /* `${...} results` */;
  emptyText.hidden = /* true when there are matches */;
}
```

✅ Hint 3

The filter test is `product.toLowerCase().includes(query)`. Show `#empty` when `matches.length` is 0. Make sure the last line of the file is `render();`.

📝 Stop here and build it

Then run `npm test -- 05-ecommerce-search-box`.

The solution, line by line

JAVASCRIPT 05/script.js (below the products array)

```
13 const searchInput = document.getElementById('search');
14 const resultsList = document.getElementById('results');
15 const countText = document.getElementById('count');
16 const emptyText = document.getElementById('empty');
17
18 function render() {
19   const query = searchInput.value.trim().toLowerCase();
20
21   const matches = products.filter((product) => {
22     return product.toLowerCase().includes(query);
23   });
24
25   resultsList.innerHTML = '';
26   matches.forEach((product) => {
27     const li = document.createElement('li');
28     li.className = 'result-item';
29     li.textContent = product;
30     resultsList.appendChild(li);
31   });
32
33   const word = matches.length === 1 ? 'result' : 'results';
34   countText.textContent = `${matches.length} ${word}`;
35
36   emptyText.hidden = matches.length > 0;
37 }
38
39 searchInput.addEventListener('input', render);
40
41 render();
```

Lines	What they do
13–16	**Select** the four elements.
19	The query: `' APPle '` becomes `'apple'`. Methods are **chained**: each one works on the result of the one before.
21–23	**Filter**: a new array with only the matching names. The original `products` is untouched, so the next search starts from all 8 again.
25	Throw away the old list. Without this line, every key press would **add** items below the old ones.
26–31	**Create** one `<li>` per match. `textContent` (not `innerHTML`) puts the name in as plain text.
33–34	The count, with correct grammar: *1 result*, *3 results* (the ternary from Day 2).
36	Hide the empty message when there are matches, show it when there are none.
39	Re-render on every change to the box. `render` without brackets: we hand the function over.
41	Render once at load, so the page doesn't start empty. The grader's first test needs this.

CSS 05/style.css

```css
.result-item {
  padding: 10px 12px;
  background: #f8fafc;
  border: 1px solid #e2e8f0;
  border-radius: 8px;
}

.result-item:hover {
  background: #ede9fe;
}

.empty {
  text-align: center;
  color: #94a3b8;
}
```

Mistakes beginners make

⚠️ **Forgetting `resultsList.innerHTML = ''`**

Type `a`, then `p`: the list shows 8 + 7 + 4 = **19** items, because each `render()` appends its matches below the old ones. Always clear before you redraw.

⚠️ **Forgetting the `render()` call at the end**

The page starts with an empty list and the first test fails (*expected 8 items, received 0*). The list only appears after you type.

⚠️ **`display: block` on `.empty`**

If you add `display: block` to `.empty`, the `hidden` property stops working, because your CSS rule beats the browser's built-in `[hidden] { display: none }`. The *No results found* text would show all the time. Don't set `display` on elements you hide with `hidden`.

⚠️ **Lower-casing only one side**

`product.includes(query.toLowerCase())` finds nothing for `apple`, because the product is `'Apple iPhone'` with a capital A. Lower-case **both**.

Interview questions

Q1 What is the difference between `forEach`, `map` and `filter`?

All three call a function for each item. `forEach` returns nothing (use it for side effects such as creating elements). `map` returns a new array of the **same length** with each item transformed. `filter` returns a new array with only the items for which the function returned `true`.

Q2 What is the difference between the `input` and `change` events?

`input` fires on every change to the value (each key press); `change` fires when the user commits the value, usually by leaving the field. Live search needs `input`.

Q3 Why use `textContent` and not `innerHTML` to show a product name?

`textContent` treats the value as plain text. `innerHTML` parses it as HTML, so text coming from users could inject tags or scripts (an **XSS** attack). Use `innerHTML` only with text you control, and `innerHTML = ''` to empty an element.

Q4 What if the list had 10,000 products coming from a server?

Rebuilding on every key press would get slow. Two common fixes: **debouncing** (wait until the user stops typing for about 300 ms, then search once), and asking the server to filter and return only the first few matches.

Practice twist

1. Add a **Clear** button: `<button id="clear" type="button">Clear</button>` after the input. On click, set the input's value to `''` and call `render()`.
2. Show `Showing all 8 products` when the box is empty, and `3 results` only while searching.
3. Harder: make `apple watch` (several spaces) still find *Apple Watch*. Hint: remove the spaces from **both** the query and the product name before comparing, with `.replaceAll(' ', '')`.

DAY 5 Exercise 06 To-Do List · Exercise 09 Dark Mode

JS

Building Elements and Switching Classes

TODAY YOU WILL LEARN

- Building an element **with children**: an `<li>` with a text and two buttons
- `classList`: `add`, `remove`, `toggle` and `contains`
- Removing an element with `remove()`
- Leaving a function early with `return`
- Why each Delete button deletes **its own** item: **closures**
- **CSS variables** and a whole dark theme from one class

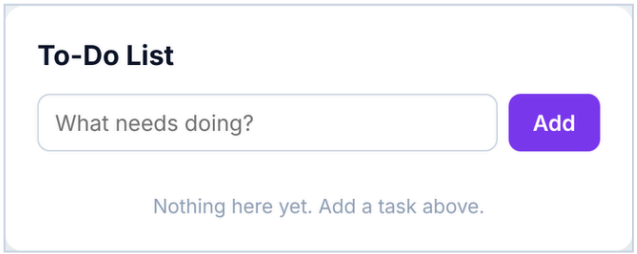
💻 **Today's plan (2 hours)**

Exercise 06 (75 min): new ideas 20 · build 40 · grade and review 15. **Exercise 09** (45 min): new ideas 10 · build 20 · grade and review 15.

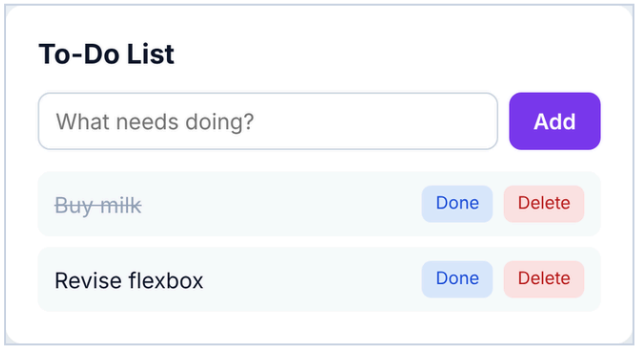
Exercise 06 · To-Do List

What you are building

START · the starter files



GOAL · after your code



Type a task and press **Add** (or Enter): it appears in the list with a **Done** button (crosses it out) and a **Delete** button (removes it). A message shows while the list is empty. The grader checks:

Test	What it does	Expects
shows the empty message until a task is added	adds *Buy milk*	`#empty` visible before, hidden after; 1 `.todo-item` containing *Buy milk*
ignores empty submissions	submits three spaces	0 items
toggling marks a task as completed	clicks the item's `.todo-toggle`	the `<li>` has class `completed`
deleting removes the task	adds two, deletes the first	1 item left, containing *Task two*
empty message returns when all tasks are deleted	adds one, deletes it	`#empty` visible

New ideas

1. An element with children

Yesterday each `<li>` held only text. Today each one holds **three children**, so you create four elements and put three inside the fourth:

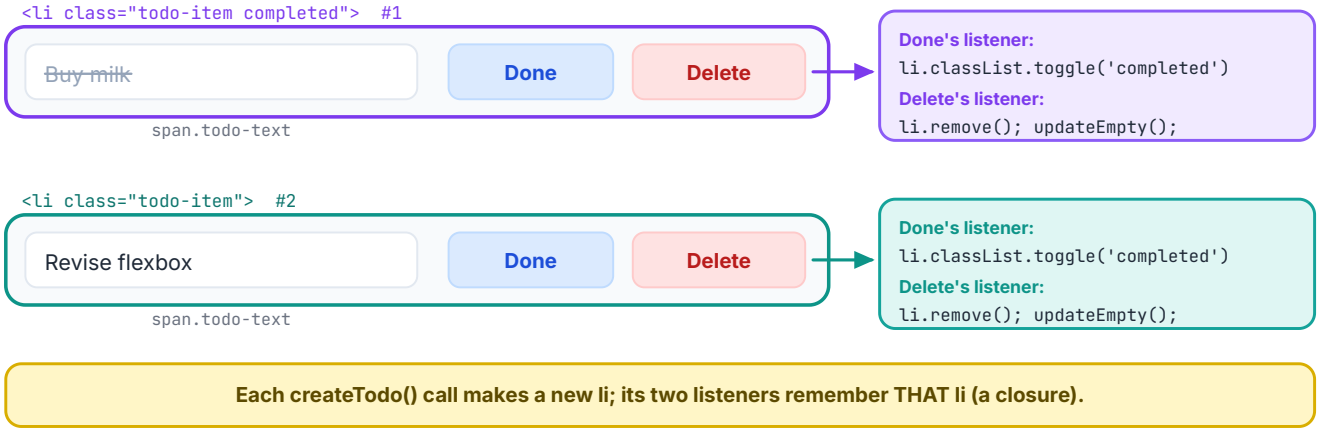


Figure 5.1 *One to-do item. `createTodo()` builds the `<li>` and its three children, then hands the `<li>` back. Each button's listener remembers **this** `<li>`.*

JAVASCRIPT children go in before the parent goes on the page

```
const li = document.createElement('li');
const span = document.createElement('span');
span.textContent = 'Buy milk';
li.appendChild(span);            // span goes inside li
list.appendChild(li);            // li goes inside the list: now visible
```

2. `classList`

Changing classes is how JavaScript and CSS work together: **JavaScript decides the state, CSS decides how each state looks**.

Code	Does
`el.classList.add('completed')`	adds the class (nothing happens if it's already there)
`el.classList.remove('completed')`	removes it
`el.classList.toggle('completed')`	adds it if missing, removes it if present
`el.classList.contains('completed')`	`true` or `false`: is it there?

`classList` changes **one** class and leaves the others alone. `el.className = 'x'` replaces **all** of them, which is fine for a new element but dangerous for an existing one.

3. `remove()` and counting children

`li.remove()` takes the element off the page. `list.children.length` counts the elements currently inside the list, which tells you if it's empty.

4. Leaving early with `return`

JAVASCRIPT a guard at the top of the handler

```
const text = input.value.trim();
if (text === '') {
  return;          // stop here: nothing to add
}
// ... only runs when there is text
```

`return` ends the function immediately. Checking for bad input at the top and returning is called a **guard**, and it keeps the rest of the function simple.

5. Functions that give something back

`createTodo(text)` will **build** an item and `return` it, without putting it on the page. The caller decides where it goes: `list.appendChild(createTodo(text))`. Building and placing are separate jobs.

6. Closures: each button remembers its own item

Inside `createTodo`, the listener for the Delete button uses the variable `li`. Every call of `createTodo` makes a **new** `li`, and each listener **remembers the** `li` **that existed when it was created**, even after `createTodo` has finished. That's why Delete on the third item deletes the third item, with no ids and no searching. A function that remembers variables from where it was written is called a **closure**.

Read the starter files

File	What is there	What you will use
`index.html`	complete	`form#todo-form`, `input#todo-input`, `ul#todo-list`, `p#empty` (visible)
`style.css`	four empty rules	`.todo-item`, `.todo-item.completed .todo-text`, `.todo-toggle`, `.todo-delete`
`script.js`	comments only	everything

The class names the grader looks for are **`todo-item`**, **`todo-text`**, **`todo-toggle`** and **`todo-delete`**. Spell them exactly.

The plan in plain English

1. Select the form, the input, the list and the empty message.
2. Write `updateEmpty()`: hide `#empty` when the list has children, show it otherwise.
3. Write `createTodo(text)` that builds and returns:
   - an `<li class="todo-item">` containing
   - a `<span class="todo-text">` with the text,
   - a `<button class="todo-toggle">Done</button>` whose click toggles `completed` on the `<li>`,
   - a `<button class="todo-delete">Delete</button>` whose click removes the `<li>` and calls `updateEmpty()`.
4. On the form's `submit`: prevent the reload; trim the text; **return** if it's empty; append `createTodo(text)` to the list; clear the input; `updateEmpty()`.
5. CSS: the item is a flex row with a light background; `.completed .todo-text` is crossed out and grey; the buttons get two different colours.

✅ Hint 1

Get the **add** part working first, with only the `<span>` inside the `<li>`. Add the buttons once you see items appear.

✅ Hint 2

Give the new buttons `type = 'button'`. A `<button>` inside a form is a **submit** button by default, so without it, clicking Done would also submit the form.

✅ Hint 3

JAVASCRIPT the Delete button's listener

```
deleteBtn.addEventListener('click', () => {
  li.remove();
  updateEmpty();
});
```

📝 Stop here and build it

Then run `npm test -- 06-todo-list`.

The solution, line by line

JAVASCRIPT 06/script.js

```
1  const form = document.getElementById('todo-form');
2  const input = document.getElementById('todo-input');
3  const list = document.getElementById('todo-list');
4  const emptyText = document.getElementById('empty');
5
6  function updateEmpty() {
7    emptyText.hidden = list.children.length > 0;
8  }
9
10 function createTodo(text) {
11   const li = document.createElement('li');
12   li.className = 'todo-item';
13
14   const span = document.createElement('span');
15   span.className = 'todo-text';
16   span.textContent = text;
17
18   const toggleBtn = document.createElement('button');
19   toggleBtn.type = 'button';
20   toggleBtn.className = 'todo-toggle';
21   toggleBtn.textContent = 'Done';
22   toggleBtn.addEventListener('click', () => {
23     li.classList.toggle('completed');
24   });
25
26   const deleteBtn = document.createElement('button');
27   deleteBtn.type = 'button';
28   deleteBtn.className = 'todo-delete';
29   deleteBtn.textContent = 'Delete';
30   deleteBtn.addEventListener('click', () => {
31     li.remove();
32     updateEmpty();
33   });
34
35   li.appendChild(span);
36   li.appendChild(toggleBtn);
```

```
  li.appendChild(deleteBtn);
  return li;
}

form.addEventListener('submit', (event) => {
  event.preventDefault();

  const text = input.value.trim();
  if (text === '') {
    return;
  }

  list.appendChild(createTodo(text));
  input.value = '';
  updateEmpty();
});

updateEmpty();
```

Lines	What they do
6–8	One small function for the empty message, used in two places (after adding and after deleting).
10–39	`createTodo` builds one complete item and returns it. Nothing appears on screen until line 49.
22–24	**Done:** toggles `completed` on the `<li>`. CSS turns that class into a line-through.
30–33	**Delete:** removes this `<li>` (the closure) and updates the empty message.
41–52	**Submit:** cancel the reload, read and trim, guard against empty text, add the item, clear the input, update the message.
54	Set the message correctly at load.

CSS 06/style.css

```css
.todo-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 8px;
  padding: 10px 12px;
  background: #f8fafc;
  border-radius: 9px;
}

.todo-item.completed .todo-text {
  text-decoration: line-through;
  color: #94a3b8;
}
```

Read the second selector from right to left: *"a* `.todo-text` *that is inside a* `.todo-item` *which* ***also*** *has the class* `completed` *"*. JavaScript only adds a class to the `<li>`; this selector reaches down to the text. (The two button rules just set `background` and `color`.)

Mistakes beginners make

⚠️ Toggling the class on the wrong element

If you toggle `completed` on the `<span>`, the selector `.todo-item.completed .todo-text` no longer matches, and the grader, which looks for the class on the `<li>`, fails. Read the task: *"toggles a* `completed` *class on the* `<li>` *"*.

⚠️ Selecting the item with `getElementById` inside the listener

Ids must be unique, and you'd need a new id for every item. The closure already gives you the right `li`; use it.

⚠️ Forgetting `updateEmpty()` after a delete

The grader's last test deletes the only task and expects the empty message back. Every place that changes the list must update the message.

Interview questions

Q1 What is a closure? Give an example from your code.

A function that remembers the variables of the place where it was created. In the to-do list, each Delete button's listener remembers the `li` created in the same `createTodo` call, so it always removes its own item.

Q2 What is the difference between `classList.toggle` and `className`?

`classList` adds, removes or toggles **one** class and keeps the others. Assigning `className` replaces the entire class list.

Q3 How would you handle clicks if there were 1,000 items?

Event delegation: one listener on the `<ul>` instead of two per item. Clicks bubble up from the button to the list; in the listener, `event.target` is the element that was clicked, and `event.target.closest('li')` finds its item.

Q4 Why `type="button"` on buttons created inside a form?

A `<button>` in a form defaults to `type="submit"`, so clicking it would submit the form. `type="button"` makes it an ordinary button.

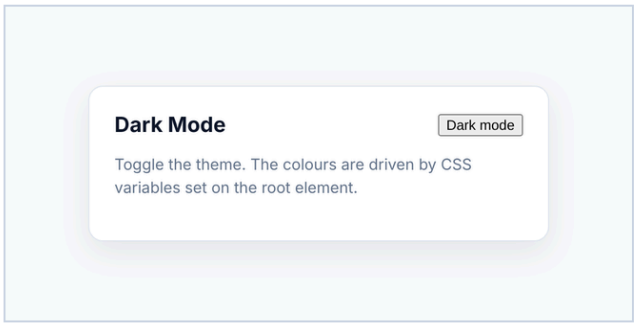
Practice twist

Show the number of tasks still to do above the list: *2 tasks left*. Hint: `list.querySelectorAll('.todo-item:not(.completed)').length`. Update it everywhere you call `updateEmpty()`.

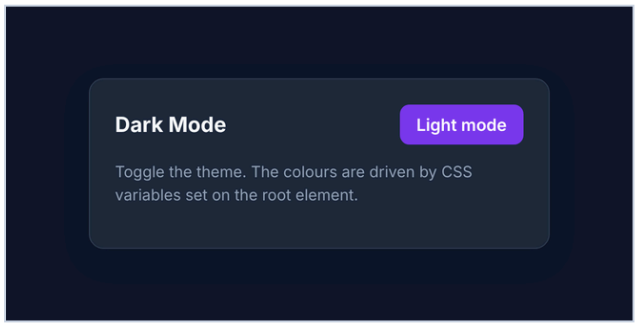
Exercise 09 · Dark-Mode Toggle

What you are building

START · the starter files



GOAL · after your code



One button switches the whole page between light and dark. The grader checks that the page starts light (`body` has no `dark` class, `aria-pressed="false"`), that one click adds `dark` to `<body>`, sets `aria-pressed="true"` and **changes the background colour**, and that a second click undoes all three.

New ideas

1. CSS variables

The starter CSS doesn't write colours directly. It defines them **once** as **custom properties** on `:root` (the `<html>` element) and uses them with `var()`:

CSS from the starter style.css

```css
:root {
  --bg: #f8fafc;
  --text: #0f172a;
}

body {
  background: var(--bg);
  color: var(--text);
}
```

A custom property's name starts with two dashes. Like `color` or `font`, it is **inherited**: every element inside reads the value from its closest ancestor that sets it.

2. A theme is just new values for the same variables

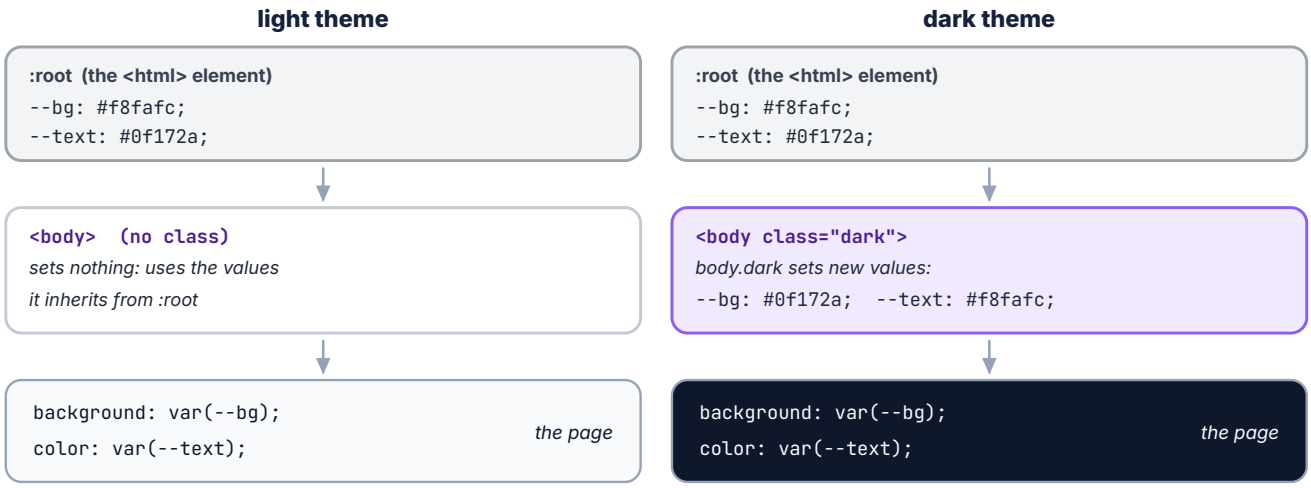


Figure 5.2 `:root` *sets the light values. When* `<body>` *gets the class* `dark`*, the rule* `body.dark` *sets* ***new values*** *for the same variables. Everything inside* `<body>` *uses the closer, dark values, so the whole page changes from one class.*

So the CSS job is **one rule**, `body.dark { ... }`, giving each variable a dark value. And the JavaScript job is **one line**, adding or removing the class.

3. `toggle` tells you what it did

`document.body` is the `<body>` element (no need to select it). `classList.toggle('dark')` adds or removes the class, and **returns** `true` if the class is now present, `false` if not. You can also ask afterwards with `classList.contains('dark')`.

The plan in plain English

1. CSS: in `body.dark { }`, give `--bg`, `--surface`, `--text`, `--muted` and `--border` dark-theme values (dark backgrounds, light text).
2. CSS: style `#theme-toggle` as a real button.
3. JS: on click of `#theme-toggle`, toggle `dark` on `document.body`; then set `aria-pressed` to `'true'` or `'false'`, and change the label to *Light mode* / *Dark mode*.

> ✅ **Hint**
>
> `setAttribute` needs a string. `String(true)` is `'true'`.

> 📝 **Stop here and build it**
>
> Then run `npm test -- 09-dark-mode-toggle`.

The solution, line by line

CSS 09/style.css

```css
body.dark {
  --bg: #0f172a;
  --surface: #1e293b;
  --text: #f8fafc;
  --muted: #94a3b8;
  --border: #334155;
}
```

JAVASCRIPT 09/script.js

```javascript
const themeBtn = document.getElementById('theme-toggle');

themeBtn.addEventListener('click', () => {
  document.body.classList.toggle('dark');

  const isDark = document.body.classList.contains('dark');
  themeBtn.setAttribute('aria-pressed', String(isDark));
  themeBtn.textContent = isDark ? 'Light mode' : 'Dark mode';
});
```

Lines	What they do
4	Switch the theme. CSS does the rest through the variables.
6	Ask the page which state we're in now.
7	Keep the ARIA state in sync: `'true'` when dark.
8	The button offers the **other** mode.

The repository's `solution/` saves one line by using the value `toggle` returns: `const isDark = document.body.classList.toggle('dark');`.

Mistakes beginners make

⚠️ **Writing `.dark` instead of `body.dark`**

`.dark { ... }` would also work here, but the task asks for `body.dark`. Read the selector you're given and keep it.

⚠️ **Changing colours one by one in JavaScript**

`document.body.style.background = '#0f172a'` for every element is long, easy to get wrong, and hard to undo. Toggle a class and let CSS hold the colours.

Interview questions

Q5 What are CSS custom properties, and why use them for themes?

Variables in CSS (`--name: value;` , read with `var(--name)`). They are inherited and can be changed per element or class, so a theme only overrides the variables and every rule that uses them updates at once.

Q6 How would you remember the user’s choice after a page reload?

Save it with `localStorage.setItem('theme', 'dark')` when it changes, and on page load read `localStorage.getItem('theme')` and add the class if it says `dark` . (Bonus exercise 19 practises `localStorage` .)

Practice twist

Start in dark mode when the user’s computer is set to dark: `window.matchMedia('(prefers-color-scheme: dark)').matches` is `true` in that case. Remember to set `aria-pressed` and the label correctly at load too.

DAY 6 Exercise 07 · else if · regular expressions · a flag

JS

Checking What the User Typed: Form Validation

TODAY YOU WILL LEARN

- Why the form says `novalidate`, and what that means for you
- Chains of decisions with `else if`
- Reading a **regular expression** for emails, and testing text with `.test()`
- The length of a string, and the **not** operator `!`
- A **flag** variable that remembers "was anything wrong?"

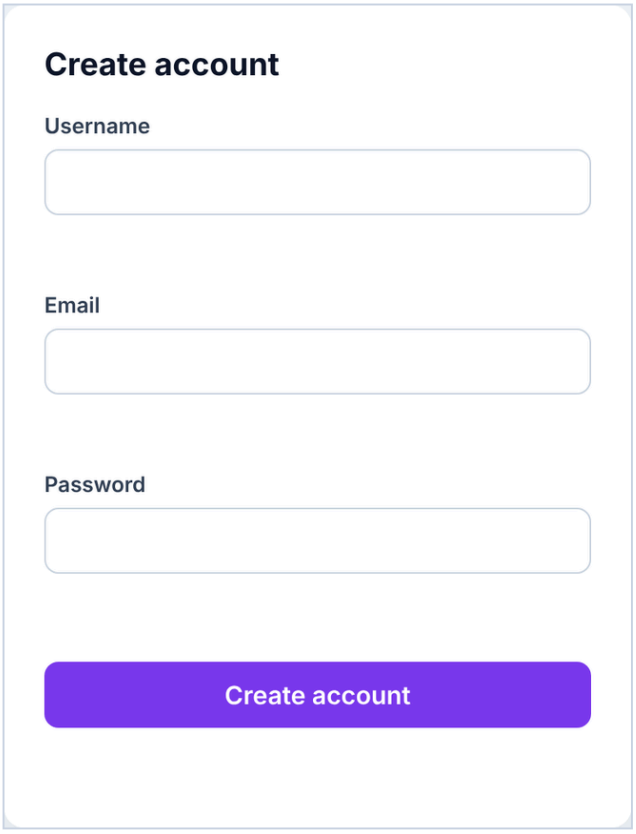
💻 **Today's plan (2 hours)**

One exercise with a lot of `if`s. **0–25** new ideas · **25–35** starter files · **35–90** build · **90–100** grade · **100–120** review. If you finish early, redo **Exercise 05** from the starter files with a 25-minute timer.

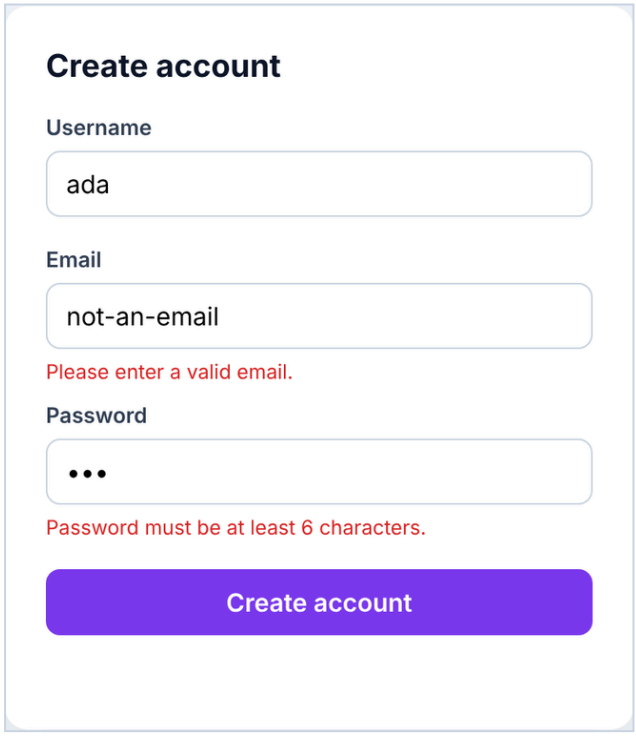
Exercise 07 · Form Validation

What you are building

START · the starter files



GOAL · after your code



On **Create account**, each field is checked. A problem shows a red message under that field; if everything is fine, a green success message appears instead. The grader checks:

Test	Fills in	Expects
blocks an empty submission and shows all errors	nothing	all three `.error` paragraphs have text; `#form-msg` is empty
flags an invalid email format	`ada` · `not-an-email` · `secret1`	`#email-error` has text; `#form-msg` empty
flags a short password	`ada` · `ada@example.com` · `123`	`#password-error` has text
accepts a valid submission	`ada` · `ada@example.com` · `secret1`	`#form-msg` has text; all three errors are empty

New ideas

1. `novalidate`: you're in charge

The form tag is `<form id="signup" class="card" novalidate>`. `novalidate` switches **off** the browser's built-in checks (the little bubbles you saw on Day 2), so the `submit` event always reaches your code and **you** decide what is valid and what message to show. Companies do this to control the wording and the look of errors.

2. `else if`: more than two choices

JAVASCRIPT checking a field in order

```
if (emailValue === '') {
  // empty
} else if (!emailPattern.test(emailValue)) {
  // not empty, but the wrong shape
} else {
  // fine
}
```

The browser tries each condition **from the top** and runs only the **first** block whose condition is true. Order matters: check *empty* before *wrong format*, so an empty field says *"Email is required"*, not *"invalid email"*.

`!` means **not**: `!true` is `false`. So `!emailPattern.test(emailValue)` reads *"the email does **not** match the pattern"*.

3. A regular expression for emails

A **regular expression** (regex) describes the **shape** of text. The task suggests this one:

JAVASCRIPT the email pattern

```
const emailPattern = /^\S+@\S+\.\S+$/;
```

Piece	Means
`/ ... /`	the slashes start and end the regex (like quotes for a string)
`^`	the **start** of the text
`\S+`	one or more characters that are **not spaces**
`@`	an actual `@`
`\.`	an actual dot (a plain `.` means *any character*, so it needs a `\`)
`$`	the **end** of the text

In words: *some non-space characters, an* `@`*, more non-space characters, a dot, more non-space characters, and nothing else.* `.test(text)` answers `true` or `false`:

JAVASCRIPT testing the pattern

```
console.log(emailPattern.test('ada@example.com'));
console.log(emailPattern.test('not-an-email'));
console.log(emailPattern.test('ada@example'));
console.log(emailPattern.test('ada lovelace@example.com'));
console.log('secret'.length, 'abc'.length < 6);
```

OUTPUT

```
true
false ← no @
false ← no dot after the @
false ← a space breaks \S+
6 true
```

The last line: `.length` on a **string** counts its characters. That's the password check.

> 📝 **You don't need to master regex**
>
> For interviews, it's enough to read this one pattern and explain each piece. Real projects usually take a tested pattern from a library rather than writing one.

4. A flag

You need to check **all three** fields (so all three messages appear together), and then decide the overall result. Start with a variable that assumes the best, and switch it off whenever a check fails:

JAVASCRIPT the flag pattern

```
let isValid = true;
if (/* something is wrong */) {
  isValid = false;
}
// ... more checks, each one may set isValid = false
if (isValid) {
  // everything passed
}
```

Read the starter files

File	What is there	What you will use
`index.html`	complete, with `novalidate`	inputs `#username`, `#email`, `#password`; error paragraphs `#username-error`, `#email-error`, `#password-error`; `#form-msg`
`style.css`	two empty rules	`.error` (small, red) and `.success` (green)
`script.js`	comments only	everything

The plan in plain English

1. Select the form, the three inputs, the three error paragraphs and `#form-msg`. Write the email pattern as a `const`.
2. On `submit`: prevent the reload; set `isValid = true`.
3. **Username:** empty after trim? Write an error and set the flag false. Otherwise clear the error.
4. **Email:** empty? *"Email is required."* Else, doesn't match the pattern? *"Please enter a valid email."* Else clear. Set the flag false in both error cases.
5. **Password:** empty? Required. Else shorter than 6? Too short. Else clear.
6. If the flag is still true, show a success message; otherwise **clear** `#form-msg`.

✅ Hint 1

Every field must end in a known state after each submit: either an error text or `''`. That's why each `if` chain ends with an `else` that clears the message. Otherwise an old error stays on screen after the user fixes the field.

✅ Hint 2

Don't `trim()` the password: spaces can be a real part of a password. The task only says *required* and *at least 6 characters*.

✅ Hint 3

The email chain:

JAVASCRIPT the email checks

```
if (emailValue === '') {
  emailError.textContent = 'Email is required.';
  isValid = false;
} else if (!emailPattern.test(emailValue)) {
  emailError.textContent = 'Please enter a valid email.';
  isValid = false;
} else {
  emailError.textContent = '';
}
```

📝 Stop here and build it

Then run `npm test -- 07-form-validation`.

The solution, line by line

JAVASCRIPT 07/script.js

```
1 const form = document.getElementById('signup');
2 const username = document.getElementById('username');
```

```
const email = document.getElementById('email');
const password = document.getElementById('password');

const usernameError = document.getElementById('username-error');
const emailError = document.getElementById('email-error');
const passwordError = document.getElementById('password-error');
const formMsg = document.getElementById('form-msg');

const emailPattern = /^\S+@\S+\.\S+$/;

form.addEventListener('submit', (event) => {
  event.preventDefault();

  let isValid = true;

  // 1. Username: required
  if (username.value.trim() === '') {
    usernameError.textContent = 'Username is required.';
    isValid = false;
  } else {
    usernameError.textContent = '';
  }

  // 2. Email: required + must look like an email
  const emailValue = email.value.trim();
  if (emailValue === '') {
    emailError.textContent = 'Email is required.';
    isValid = false;
  } else if (!emailPattern.test(emailValue)) {
    emailError.textContent = 'Please enter a valid email.';
    isValid = false;
  } else {
    emailError.textContent = '';
  }

  // 3. Password: required + at least 6 characters
  if (password.value === '') {
    passwordError.textContent = 'Password is required.';
    isValid = false;
  } else if (password.value.length < 6) {
    passwordError.textContent = 'Password must be at least 6 characters.';
    isValid = false;
  } else {
    passwordError.textContent = '';
  }

  // 4. The overall message
  if (isValid) {
    formMsg.textContent = 'Account created successfully!';
  } else {
    formMsg.textContent = '';
  }
});
```

Lines	What they do
1–11	Select eight elements and define the pattern once, outside the listener.
14	No reload.
16	The flag: innocent until proven guilty.
19–24	Username: one check, two outcomes.
27–36	Email: three outcomes, in the right order. The value is trimmed once and reused.
39–47	Password: same shape as the email, with `.length` instead of the pattern.
50–54	The overall result. Clearing `#form-msg` on failure matters: after one successful submit, a later failed one must not still say *success*.

The repository's `solution/` puts the checks in a separate `validate()` function that **returns** `true` or `false`, and the listener calls `if (validate()) { ... }`. That's a good next step once this version makes sense: it separates *checking* from *reacting*.

Mistakes beginners make

⚠️ **Stopping at the first error with `return`**

If you `return` as soon as the username is empty, the email and password errors never appear, and the first test (all three errors at once) fails. Check everything, **then** decide.

⚠️ **Writing `.` instead of `\.` in the pattern**

`/^\S+@\S+.\S+$/` accepts `ada@examplecom`, because a plain `.` matches **any** character. Escape it: `\.`.

⚠️ **Never clearing old errors**

Without the `else { error.textContent = ''; }` branches, an error from a previous attempt stays on screen: submit an empty form, then type a username and submit again, and *Username is required.* is still there. The grader won't catch this (each test starts on a freshly loaded page), but anyone trying your page will.

Interview questions

Q1 Why validate in JavaScript if the server validates anyway?

Client-side validation gives **instant feedback** and saves a round trip. But it can be bypassed (anyone can turn off JavaScript or send a request directly), so the **server must always validate too**. Client-side is for the user's comfort, server-side is for security.

Q2 Explain `/^\S+@\S+\.\S+$/`.

Start of text, one or more non-space characters, `@`, one or more non-space characters, a literal dot, one or more non-space characters, end of text. It's a basic shape check, not a full email validator.

Q3 What does the `novalidate` attribute do?

It turns off the browser's built-in form checks (`required`, `type="email"` and so on) on submit, so the page's JavaScript can handle validation and messages itself.

Practice twist

Validate **as the user types**: listen for `input` on each field and update only that field's error. Hint: move each field's check into its own function (`checkUsername()`, `checkEmail()`, `checkPassword()`) that writes the error and returns `true` or `false`. The submit listener then becomes:

JAVASCRIPT the submit listener after the twist

```
const okName = checkUsername();
const okEmail = checkEmail();
const okPassword = checkPassword();
if (okName && okEmail && okPassword) { /* success */ }
```

Why not simply `if (checkUsername() && checkEmail() && checkPassword())`? (Answer: `&&` stops at the first `false`, so the later checks wouldn't run and their errors wouldn't show.)

DAY 7 Exercise 13 Accordion · Exercise 14 Tab Switcher

JS

One Open at a Time: Accordion and Tabs

TODAY YOU WILL LEARN

- Selecting **many** elements with `querySelectorAll`, and searching **inside** an element
- The pattern behind accordions, tabs and carousels: **switch all off, then one on**
- Remembering the state **before** you change it
- `data-*` **attributes** and `dataset`: storing a name on an element
- Attribute selectors such as `.panel[data-panel="pricing"]`

💻 **Today's plan (2 hours)**

Exercise 13 (65 min): new ideas 20 · build 30 · grade and review 15. **Exercise 14** (55 min): new ideas 10 · build 30 · grade and review 15. The two exercises use the same pattern, so the second one should feel faster.

Exercise 13 · Accordion

What you are building

START · the starter files

FAQ

What is HTML? +

HTML is the standard markup language used to structure content on the web.

What is CSS? +

CSS describes how HTML elements should be displayed on screen.

What is JavaScript? +

JavaScript makes web pages interactive and dynamic.

GOAL · after your code

FAQ

What is HTML? +

What is CSS? -

CSS describes how HTML elements should be displayed on screen.

What is JavaScript? +

A FAQ list. Every answer starts hidden. Clicking a question opens its answer and closes any other; clicking an open question closes it. The `+` becomes `-` while open. The grader checks:

Test	What it does	Expects
all panels start closed	opens the page	every `.accordion-body` hidden
clicking a header opens its panel	clicks header 1	body 1 visible; header 1 has `aria-expanded="true"`
opening one panel closes the others	clicks header 1, then header 2	body 1 hidden, body 2 visible
clicking an open header closes it again	clicks header 2 twice	body 2 hidden

New ideas

1. Selecting many elements

Method	Returns
`document.getElementById('x')`	the one element with id `x` (or `null`)
`document.querySelector('.tab')`	the **first** element matching a **CSS selector** (or `null`)
`document.querySelectorAll('.tab')`	**all** matching elements, as a list (empty if none)

The list from `querySelectorAll` (a **NodeList**) has `.length`, `[0]`, `[1]`... and `forEach`, which is all you need today.

2. Searching inside one element

`querySelector` also works **on an element**, and then it only searches inside that element:

JAVASCRIPT find the header of this item, not any header

```
const header = item.querySelector('.accordion-header');
const icon = item.querySelector('.icon');
```

This is how you go from “one accordion item” to “its own header and its own icon”.

3. A listener for every item

`forEach` + `addEventListener` gives each item its own listener, and thanks to the **closure** from Day 5, each listener remembers its own `item` and `header`:

JAVASCRIPT one listener per item

```
items.forEach((item) => {
  const header = item.querySelector('.accordion-header');
  header.addEventListener('click', () => {
    // here, `item` is the item whose header was clicked
  });
});
```

4. Switch all off, then one on



The user clicked Question 3 (orange arrow). Question 2 was open. Read the state first, then switch all off, then one on.

Figure 7.1 *The pattern for "only one at a time". **1.** Remember whether the clicked item was open. **2.** Switch every item off. **3.** Switch the clicked one on (for the accordion: only if it wasn't open before, so a second click closes it).*

You never have to work out *which* item is currently open. Turning **all** of them off is simpler and can't go wrong.

The order of steps 1 and 2 matters: if you closed everything first and then asked *"was this item open?"*, the answer would always be *no*. **Read the state before you change it.**

5. Hiding with a class

CSS hides every body by default and shows a body only when its item has the class `open`:

CSS state in a class, looks in CSS

```
.accordion-body { display: none; }
.accordion-item.open .accordion-body { display: block; }
```

`display: none` removes the element from the page completely (as if it weren't there). The JavaScript only adds and removes `open`.

Read the starter files

Each FAQ entry looks like this:

HTML one item in index.html

```
<div class="accordion-item">
  <button class="accordion-header" type="button" aria-expanded="false">
    What is HTML?
    <span class="icon">+</span>
  </button>
  <div class="accordion-body">
    <p>HTML is the standard markup language ...</p>
  </div>
</div>
```

`style.css` has two empty rules: `.accordion-body` and `.accordion-item.open .accordion-body`. `script.js` has only comments.

The plan in plain English

1. CSS: hide `.accordion-body`; show it inside `.accordion-item.open`.
2. JS: select all `.accordion-item`s.
3. For each item, find its header and listen for clicks. On a click:
    1. Remember `wasOpen`: does this item have the class `open`?
    2. For **every** item: remove `open`, set its header's `aria-expanded` to `'false'`, set its icon to `+`.
    3. If it **wasn't** open: add `open` to this item, set its header's `aria-expanded` to `'true'`, its icon to `-`.

✅ **Hint 1**

Get it working without the "click again to close" rule first: switch all off, then switch the clicked one on. Then add `wasOpen`.

✅ **Hint 2**

Inside the "all off" loop, use a different name for the loop variable (`other`), so you don't mix it up with the clicked `item`.

📝 **Stop here and build it**

Then run `npm test -- 13-accordion`.

The solution, line by line

CSS 13/style.css

```css
.accordion-body {
  display: none;
}

.accordion-item.open .accordion-body {
  display: block;
}
```

JAVASCRIPT 13/script.js

```javascript
const items = document.querySelectorAll('.accordion-item');

items.forEach((item) => {
  const header = item.querySelector('.accordion-header');

  header.addEventListener('click', () => {
    const wasOpen = item.classList.contains('open');

    // 1. Close every item
    items.forEach((other) => {
      const otherHeader = other.querySelector('.accordion-header');
      other.classList.remove('open');
      otherHeader.setAttribute('aria-expanded', 'false');
      other.querySelector('.icon').textContent = '+';
    });

    // 2. Open the clicked one, unless it was already open
    if (!wasOpen) {
      item.classList.add('open');
      header.setAttribute('aria-expanded', 'true');
      item.querySelector('.icon').textContent = '-';
    }
  });
});
```

Lines	What they do
1	All three items, in page order.
3–4	For each item, find **its** header once.
6	One listener per header.
7	Read the state **before** changing anything.
10–15	All off: class, ARIA and icon, for every item (the clicked one included).
18–22	One on, but only if it was closed. If it was open, step 1 already closed it, which is exactly the “click again to close” behaviour.

Mistakes beginners make

⚠️ **Asking “was it open?” after closing everything**

The answer is always *no*, so a second click re-opens the item instead of closing it, and the last test fails.

⚠️ `document.querySelector('.icon')` **inside the loop**

`document.querySelector` searches the **whole page** and always returns the **first** icon, so only the first `+` ever changes. Search inside the item: `other.querySelector('.icon')`.

Interview questions

Q1 **What is the difference between `querySelector` and `querySelectorAll`?**

`querySelector` returns the first element that matches a CSS selector (or `null`); `querySelectorAll` returns all matches as a NodeList, which you can loop over with `forEach`.

Q2 **What is the difference between `display: none` and `visibility: hidden`?**

`display: none` removes the element from the layout: it takes no space. `visibility: hidden` makes it invisible but keeps its space.

Q3 **How would you let several panels be open at once?**

Remove the “close every item” loop and just toggle the clicked one: `item.classList.toggle('open')`, then set `aria-expanded` and the icon from the result.

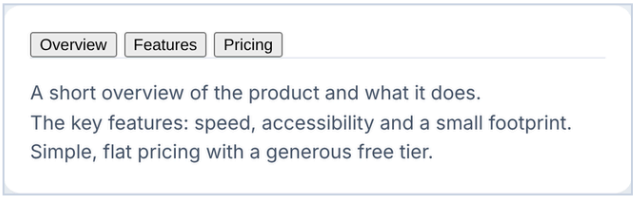
Practice twist

Animate the opening. Instead of `display`, use `max-height: 0; overflow: hidden; transition: max-height 0.3s;` on `.accordion-body`, and `max-height: 200px` when open. Note that the grader checks visibility, so it may fail with this version. That’s fine for a twist; keep a copy of the passing version.

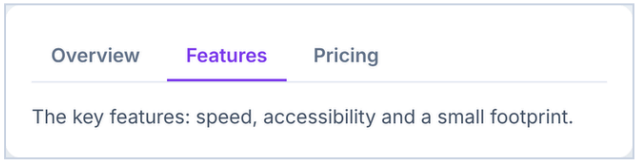
Exercise 14 · Tab Switcher

What you are building

START · the starter files



GOAL · after your code



Three tabs; clicking one underlines it and shows its panel. The grader checks that on load the first tab is `active` and only the *overview* panel is visible; that clicking *Features* shows its panel and hides *overview*; and that after clicking *Pricing*, that tab has `active` and `aria-selected="true"` while *Overview* has neither.

New ideas

1. `data-*` attributes

How does a tab know which panel is its own? The HTML **labels** both with the same word:

HTML from index.html

```
<button class="tab" data-tab="features" ...>Features</button>
...
<div class="panel" data-panel="features">...</div>
```

Any attribute starting with `data-` is yours to invent: it stores extra information on an element. JavaScript reads it through `dataset`, without the `data-` prefix:

JAVASCRIPT reading a data attribute

```
tab.dataset.tab        // 'features'   (from data-tab)
panel.dataset.panel    // 'features'   (from data-panel)
```

A name with a dash such as `data-panel-id` becomes **camelCase** in `dataset`: `dataset.panelId`.

2. Attribute selectors

CSS (and so `querySelector`) can select by **attribute value**:

JAVASCRIPT find the panel with a given name

```
const panel = document.querySelector(`.panel[data-panel="${name}"]`);
```

With `name` equal to `'pricing'`, the selector becomes `.panel[data-panel="pricing"]`. The backticks build the selector text, exactly as they built messages on Day 2.

Read the starter files

`index.html` is complete: three `.tab` buttons (the first already has `active` and `aria-selected="true"`) and three `.panel`s (the first is `active`). In `style.css`, four rules are empty: `.tab`, `.tab.active`, `.panel` and `.panel.active`. `script.js` has only comments.

The plan in plain English

1. CSS: plain tabs (no border except a transparent bottom line, transparent background, pointer); the active tab coloured with a coloured bottom border; `.panel` hidden, `.panel.active` shown.
2. JS: select all tabs and all panels.
3. For each tab, on click:
    1. Read its name from `dataset.tab`.

2. All off: remove `active` from every tab (and set `aria-selected="false"`) and from every panel.
3. One on: add `active` to the clicked tab (`aria-selected="true"`) and to the panel whose `data-panel` equals the name.

> ✅ **Hint**
>
> This is the accordion's pattern without `wasOpen`: a tab can't be "closed" by clicking it again. Write it from memory before looking back.

> 📝 **Stop here and build it**
>
> Then run `npm test -- 14-tab-switcher`.

The solution, line by line

CSS 14/style.css

```css
.tab {
  padding: 10px 16px;
  border: none;
  border-bottom: 2px solid transparent;
  background: transparent;
  color: #64748b;
  font: inherit;
  font-weight: 600;
  cursor: pointer;
}

.tab.active {
  color: #7c3aed;
  border-bottom-color: #7c3aed;
}

.panel {
  display: none;
}

.panel.active {
  display: block;
}
```

Every tab gets a **transparent** 2px bottom border, and the active one only changes its colour. That way the text doesn't jump down by 2px when a tab becomes active.

JAVASCRIPT 14/script.js

```
1  const tabs = document.querySelectorAll('.tab');
2  const panels = document.querySelectorAll('.panel');
3
4  tabs.forEach((tab) => {
5    tab.addEventListener('click', () => {
6      const name = tab.dataset.tab;
7
8      // 1. Switch every tab and every panel off
9      tabs.forEach((t) => {
10       t.classList.remove('active');
11       t.setAttribute('aria-selected', 'false');
12     });
13     panels.forEach((panel) => {
14       panel.classList.remove('active');
15     });
16
17     // 2. Switch on the clicked tab and the panel with the same name
18     tab.classList.add('active');
19     tab.setAttribute('aria-selected', 'true');
20     const panel = document.querySelector(`.panel[data-panel="${name}"]`);
21     panel.classList.add('active');
22   });
23 });
```

Lines	What they do
1–2	All tabs and all panels.
6	Which panel do we want? Read the name stored on the tab.
9–15	All off.
18–19	The clicked tab on.
20–21	Find the panel with the same name and switch it on.

🔍 Another way: `toggle` with a second argument

`classList.toggle('active', condition)` **adds** the class when the condition is `true` and **removes** it when `false`. The repository's `solution/` uses it to do "all off, one on" in a single loop:

JAVASCRIPT exercises/14-tab-switcher/solution/script.js (part)

```
tabs.forEach((t) => {
  const isActive = t === tab;
  t.classList.toggle('active', isActive);
  t.setAttribute('aria-selected', String(isActive));
});

panels.forEach((panel) => {
  panel.classList.toggle('active', panel.dataset.panel === name);
});
```

Mistakes beginners make

⚠️ `tab.dataset['data-tab']` or `tab.dataset.dataTab`

`dataset` drops the `data-` prefix: `data-tab` is `dataset.tab`.

⚠️ Using the tab's index to find the panel

`panels[i]` works only while the tabs and panels are in the same order. The `data-*` names keep working even if someone reorders the HTML. That robustness is the point of the exercise.

Interview questions

Q4 What are `data-*` attributes for?

Storing custom information on HTML elements, such as an id, a name or a state, which JavaScript reads through `element.dataset`. They're valid HTML and don't affect how the page looks.

Q5 Why use classes to show and hide, instead of setting `style.display` in JavaScript?

It keeps the looks in CSS and the logic in JavaScript. One class can change many properties, and the CSS can change (for example to add an animation) without touching the script.

Q6 What ARIA attributes does a tab component need?

`role="tablist"` on the container, `role="tab"` and `aria-selected` on each tab, and `role="tabpanel"` on each panel. The starter HTML has the first two; you keep `aria-selected` in sync.

Practice twist

Make the arrow keys work: when a tab has focus, → activates the next tab and ← the previous one. Hint: listen for `keydown` on each tab; `event.key` is `'ArrowRight'` or `'ArrowLeft'`. Move the activation code into a function `activate(tab)` so the click and the key both call it.

DAY 8 Exercise 16 Image Carousel · Exercise 17 Live Clock ⭐

JS

Counting in Circles, and Time

TODAY YOU WILL LEARN

- Keeping **which slide is showing** in a number, and picking an element by index
- **Wrap-around** with the remainder operator `%` (and its trap with negative numbers)
- Turning text into a number with `Number()`
- The `Date` object: the current hours, minutes and seconds
- **Zero-padding** with `padStart`, and running code **every second** with `setInterval`

💻 **Today's plan (2 hours)**

Exercise 16 (65 min): new ideas 20 · build 30 · grade and review 15. **Exercise 17** (55 min): new ideas 15 · build 25 · grade and review 15. The clock is the second question reported from the real round: after finishing it, close the file, reopen the starter (Day 10 shows how) and build it again in **under 15 minutes**.

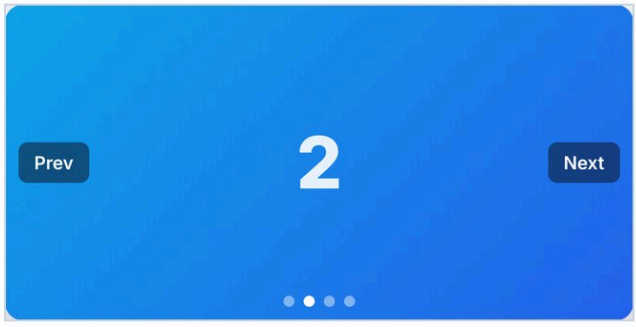
Exercise 16 · Image Carousel

What you are building

START · the starter files



GOAL · after your code



Four coloured slides, one visible at a time. **Next** and **Prev** move through them, the dots jump to a slide, and the ends **wrap around**: Next on slide 4 goes to slide 1, Prev on slide 1 goes to slide 4. The grader checks:

Test	What it does	Expects
shows the first slide and dot on load	opens the page	slide 1 and dot 1 have `active`; exactly one `.slide.active`
Next and Prev move between slides	Next, then Prev	slide 2 active (and slide 1 not), then slide 1 again
dots jump to a specific slide	clicks dot 3	slide 3 and dot 3 active
navigation wraps around at the ends	dot 4, Next, Prev	slide 1, then slide 4

New ideas

1. A number as the state

The whole carousel is described by **one number**: the index of the slide being shown, from 0 to 3. Keep it in `let index = 0;`. Every button only changes that number and then **shows** the slide it points to: `slides[index]` is the slide element at that position, just like an array.

2. Wrap-around with `%`

`%` gives the **remainder** of a division: `5 % 4` is 1, because 5 = 4 + **1**. With 4 slides, `% 4` folds every number back into 0…3:

JAVASCRIPT remainders with 4 slides

```
const n = 4; // number of slides
console.log(1 % n, 3 % n, 4 % n, 5 % n);
console.log(-1 % n);
console.log((-1 + n) % n, (4 + n) % n, (2 + n) % n);
```

OUTPUT

```
1 3 0 1
-1 ← the trap
3 0 2 ← the fix: add n first
```

So **Next** from the last slide (3 + 1 = 4) gives `4 % 4` = **0**: back to the first. But **Prev** from the first slide (0 − 1 = −1) gives **−1** in JavaScript, not 3, and `slides[-1]` doesn't exist. Adding `n` before taking the remainder fixes both directions: `(i + n) % n`.

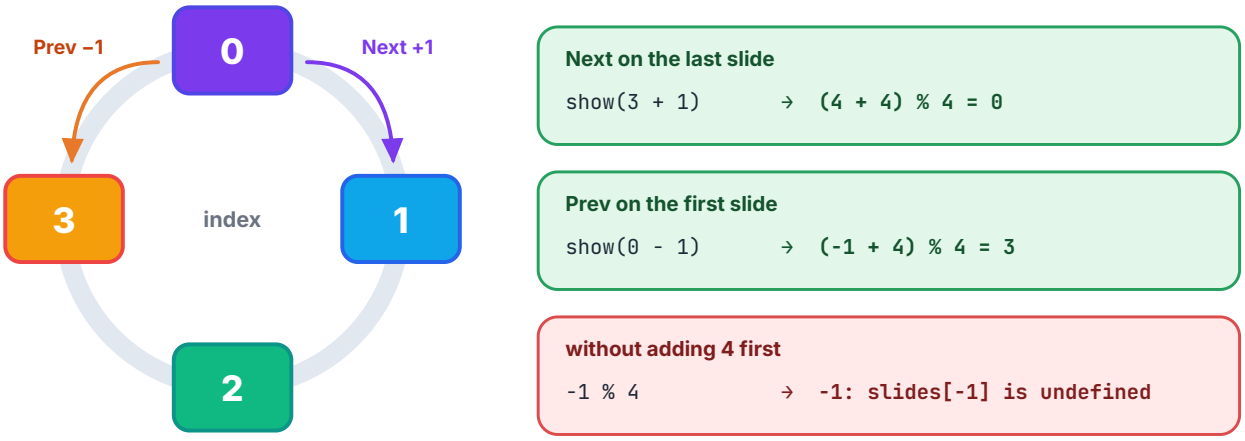


Figure 8.1 *Four slides in a circle. Next adds 1 and Prev subtracts 1; `(i + 4) % 4` turns 4 into 0 and −1 into 3, so you can never fall off either end.*

3. One function for every button

Next, Prev and the four dots all do the same thing with a different target, so write it **once**:

JAVASCRIPT the shape of the carousel

```
function show(i) {
  // 1. wrap i into 0..3 and store it in index
  // 2. all slides and dots off
  // 3. slide[index] and dot[index] on
}
nextBtn.addEventListener('click', () => show(index + 1));
```

`() => show(index + 1)` is a short arrow function: when clicked, call `show` with one more than the current index. It's the "all off, one on" pattern from Day 7, chosen by a number instead of a click.

4. Text into numbers

Each dot stores its position in `data-index="2"`. Attribute values are **text**, and `+` on text **joins** instead of adding:

JAVASCRIPT text versus numbers

```
console.log(Number('2') + 1, '2' + 1);
```

OUTPUT

```
3 21
```

So turn the value into a number first: `Number(dot.dataset.index)`.

5. CSS: the later rule wins

The starter CSS has `.slide { display: grid; ... }` near the top and an empty `.slide { }` further down, where you write `display: none`. Both selectors have the same score, so the **later** one wins and all slides hide. `.slide.active` scores higher (two classes), so you give it `display: grid` back for the one active slide.

Read the starter files

File	What is there	What you will use
`index.html`	complete	four `.slide`s (the first is `active`), `#prev`, `#next`, four `.dot`s with `data-index` 0–3
`style.css`	four empty rules	`.slide`, `.slide.active`, `.dot`, `.dot.active`
`script.js`	comments only	everything

The plan in plain English

1. CSS: hide `.slide`, show `.slide.active` with `display: grid` (to keep the number centered). Dots: 10px circles, half-transparent white; the active dot solid white.
2. JS: select all slides, all dots, Prev and Next. `let index = 0;`
3. `show(i)`: `index = (i + slides.length) % slides.length`; remove `active` from all slides and dots; add it to `slides[index]` and `dots[index]`.
4. Next → `show(index + 1)`; Prev → `show(index - 1)`; each dot → `show(Number(dot.dataset.index))`.
5. Call `show(0)` at the end.

> ✅ **Hint 1**
>
> Use `slides.length` instead of writing `4`. Add a fifth slide to the HTML later and the code still works.

> ✅ **Hint 2**
>
> If Prev on the first slide makes everything disappear, you have the negative remainder problem: `index` became −1.

> 📝 **Stop here and build it**
>
> Then run `npm test -- 16-image-carousel`.

The solution, line by line

CSS 16/style.css

```css
.slide {
  display: none;
}

.slide.active {
  display: grid;
}
```

CSS 16/style.css (the dots)

```css
.dot {
  width: 10px;
  height: 10px;
  padding: 0;
  border: none;
  border-radius: 50%;
  background: rgba(255, 255, 255, 0.45);
  cursor: pointer;
}

.dot.active {
  background: #ffffff;
}
```

JAVASCRIPT 16/script.js

```
1  const slides = document.querySelectorAll('.slide');
2  const dots = document.querySelectorAll('.dot');
3  const prevBtn = document.getElementById('prev');
4  const nextBtn = document.getElementById('next');
5
6  let index = 0;
7
8  function show(i) {
9    // wrap around: -1 becomes the last slide, 4 becomes 0
10   index = (i + slides.length) % slides.length;
11
12   slides.forEach((slide) => slide.classList.remove('active'));
13   dots.forEach((dot) => dot.classList.remove('active'));
14
15   slides[index].classList.add('active');
16   dots[index].classList.add('active');
17 }
18
19 nextBtn.addEventListener('click', () => show(index + 1));
20 prevBtn.addEventListener('click', () => show(index - 1));
21
22 dots.forEach((dot) => {
23   dot.addEventListener('click', () => show(Number(dot.dataset.index)));
24 });
25
26 show(0);
```

Lines	What they do
6	The state: which slide is showing.
10	Wrap the requested position into 0…3 and **store** it. Next and Prev compute from this stored value.
12–13	All off (short arrow functions with one statement need no braces).
15–16	One on: the slide and the dot at the same position.
19–20	Next and Prev only say **where to go**; `show` does the rest.
22–24	Each dot jumps to its own position, converted from text to a number.
26	Make the page consistent at load.

Mistakes beginners make

⚠️ `index = i % slides.length`

Next works, but Prev on the first slide gives −1, `slides[-1]` is `undefined`, and `undefined.classList` throws *Cannot read properties of undefined*. Add the length first.

⚠️ **Forgetting to store the new index**

`show(index + 1)` with a `show` that never sets `index` moves to slide 2 and then stays there, because `index` is still 0 every time.

Interview questions

Q1 How do you make a carousel wrap around?

Keep the current index and compute the next one with `(i + n) % n`, where `n` is the number of slides. Adding `n` first handles going backwards from 0, because `%` in JavaScript keeps the sign of the left number.

Q2 How would you make it play automatically, and pause on hover?

`const timer = setInterval(() => show(index + 1), 3000);` starts it. On `mouseenter`, `clearInterval(timer)`; on `mouseleave`, start a new interval.

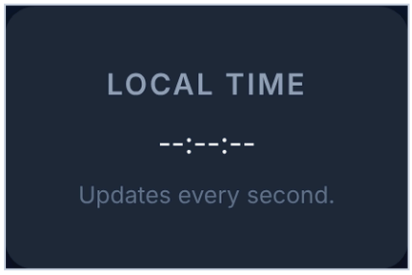
Practice twist

Add **autoplay**: move to the next slide every 3 seconds (`setInterval`, which you'll learn in a minute), and stop when the mouse is over the carousel.

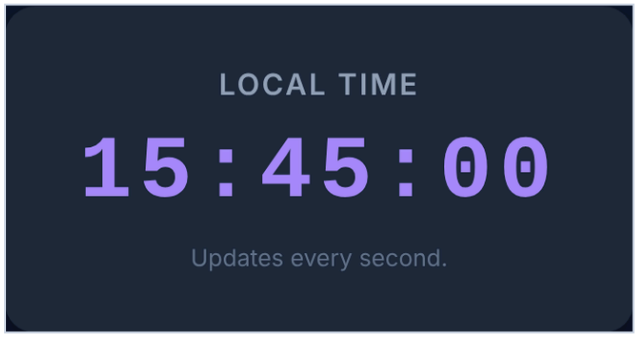
Exercise 17 · Live Digital Clock ⭐

What you are building

START · the starter files



GOAL · after your code



A clock showing the current time as `HH:MM:SS`, updating every second. The grader checks that `#clock` matches the shape two digits, colon, two digits, colon, two digits; that the hours and minutes equal the real current time; and that the text **changes** within 3 seconds.

New ideas

1. `Date`

`new Date()` creates an object holding **this exact moment**. Ask it for parts:

JAVASCRIPT reading a date

```
const d = new Date(2026, 9, 15, 9, 5, 3); // 15 Oct 2026, 09:05:03
console.log(d.getHours(), d.getMinutes(), d.getSeconds());
```

OUTPUT

```
9 5 3
```

With no arguments, `new Date()` is **now**, in the user's own time zone. The hours go from 0 to 23. (The example builds a fixed date so the output stays the same. Note month 9 is **October**: months count from 0, days don't.)

2. Zero-padding

`9:5:3` is not `HH:MM:SS`. Each part must be two digits:

JAVASCRIPT padStart

```
console.log(String(7).padStart(2, '0'));
console.log(String(12).padStart(2, '0'));
```

OUTPUT

```
07
12
```

`padStart(2, '0')` adds `'0'`s **at the start** until the text is 2 characters long. It's a string method, so turn the number into a string first with `String(...)`. Put this in a tiny function, `pad(n)`, because you need it three times.

3. Run something every second

JAVASCRIPT setInterval and setTimeout

```
setInterval(updateClock, 1000);   // call updateClock every 1000 ms, forever
setTimeout(updateClock, 1000);    // call it ONCE, after 1000 ms
```

`setInterval` keeps calling your function. It returns an id; `clearInterval(id)` stops it. Note that you pass `updateClock` **without** brackets again: the browser calls it later.

Call it once yourself first. The interval's first call happens after a full second, so without an immediate call the page shows `--:--:--` for one second.

4. CSS for digits

In most fonts a `1` is narrower than an `8`, so the time would wobble left and right as it changes. A **monospace** font (every character the same width) keeps it still.

Read the starter files

`index.html` has `<div id="clock" class="clock">--:--:--</div>`. `style.css` has an empty `.clock` rule (large monospace text in a purple colour). `script.js` has only comments.

The plan in plain English

1. Select `#clock`.
2. Write `pad(number)`: turn it into a string and pad it to 2 characters with `'0'`.
3. Write `updateClock()`: make a `new Date()`, read the hours, minutes and seconds, pad each, and write `` `${hours}:${minutes}:${seconds}` `` into the clock.
4. Call `updateClock()` once, then `setInterval(updateClock, 1000)`.
5. CSS: monospace font, about `3rem`, bold, a purple colour, some letter spacing.

> 📝 **Stop here and build it**
>
> Then run `npm test -- 17-live-clock`. One test waits up to 3 seconds for the clock to change, so this exercise takes a moment longer to grade.

The solution, line by line

JAVASCRIPT 17/script.js

```
1  const clock = document.getElementById('clock');
2
3  function pad(number) {
4    return String(number).padStart(2, '0');
5  }
6
7  function updateClock() {
8    const now = new Date();
9    const hours = pad(now.getHours());
10   const minutes = pad(now.getMinutes());
11   const seconds = pad(now.getSeconds());
12   clock.textContent = `${hours}:${minutes}:${seconds}`;
13 }
14
15 updateClock();
16 setInterval(updateClock, 1000);
```

Lines	What they do
3–5	A helper that **returns** a value: 7 → `'07'`, 12 → `'12'`.
8	A fresh `Date` on every call. Creating it once outside the function would freeze the clock at the page's load time.
9–12	Read, pad and show.
15	Show the time immediately.
16	Then refresh it every second.

CSS 17/style.css

```css
.clock {
  font-family: Consolas, "Courier New", monospace;
  font-size: 3rem;
  font-weight: 700;
  letter-spacing: 0.08em;
  color: #a78bfa;
}
```

`font-family` is a list: the browser uses the first font it has. `monospace` at the end means *"any fixed-width font"*, so there's always a fallback.

Mistakes beginners make

⚠️ `const now = new Date()` **outside the function**

The clock shows the load time and never changes: each call reads the **same** old moment. The grader's third test fails.

⚠️ `now.getHours().padStart(2, '0')`

TypeError: now.getHours(...).padStart is not a function. `getHours()` returns a **number**, and `padStart` exists only on strings. Wrap it: `String(now.getHours())`.

⚠️ `setInterval(updateClock(), 1000)`

The brackets call `updateClock` once and pass its result (`undefined`) to `setInterval`. The clock shows the time but never updates. Same rule as Day 1: hand over the function, don't call it.

Interview questions

Q3 What is the difference between `setTimeout` and `setInterval`?

`setTimeout(fn, ms)` calls `fn` **once** after `ms` milliseconds; `setInterval(fn, ms)` calls it **repeatedly** every `ms`. Both return an id you can pass to `clearTimeout` / `clearInterval` to cancel.

Q4 **Is `setInterval(fn, 1000)` exactly one second?**

No. It's **at least** about 1000 ms: if the browser is busy, or the tab is in the background, calls can be late. That's why the clock reads `new Date()` on every call instead of adding 1 to a counter: the time shown is always correct, even if one tick was late.

Q5 **How do you show a 12-hour clock with AM/PM?**

`const suffix = h >= 12 ? 'PM' : 'AM';` and `const h12 = h % 12 || 12;` (`h % 12` turns 13 into 1, and `|| 12` turns 0 into 12).

Practice twist

Add the date under the clock, such as *Thursday, 15 October 2026*. Look up `toLocaleDateString` with the options `{ weekday: 'long', day: 'numeric', month: 'long', year: 'numeric' }`. Then add a button that switches between 24-hour and 12-hour format.

DAY 9 Exercise 15 · objects · filter + sort · state + render

JS

Data on Screen: the Dynamic Table

TODAY YOU WILL LEARN

- **Objects**: one employee as `{ name, age, city }`, and arrays of objects
- Filtering on **two fields** with `||` (or)
- **Sorting** numbers and names correctly, and the default `sort()` trap
- **State + render**: remember the choices in variables, redraw from them every time
- Building table rows with a template

💻 **Today's plan (2 hours)**

The hardest exercise in the 14, but you already know most of it from Day 4. **0–30** new ideas · **30–40** starter files · **40–95** build · **95–105** grade · **105–120** review. Tonight, read the Day 10 plan so you can start the mock round tomorrow without preparation.

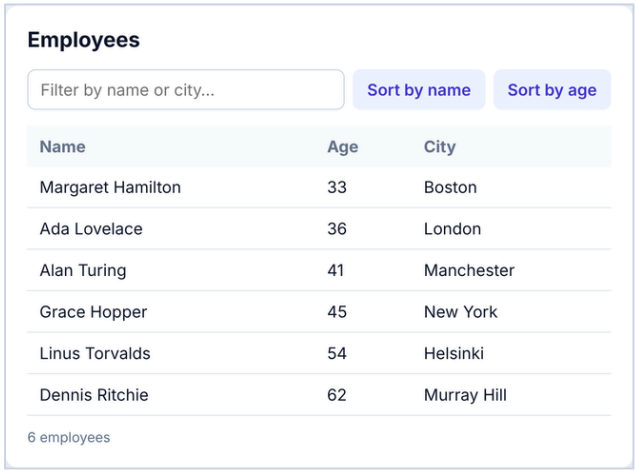
Exercise 15 · Dynamic Table

What you are building

START · the starter files



GOAL · after your code



A table of six employees, built by JavaScript from an array. A filter box keeps rows whose **name or city** contains the text, and two buttons sort by name (A–Z) or by age (youngest first). A line under the table counts the rows. The grader checks:

Test	What it does	Expects
renders all rows on load	opens the page	6 `<tr>` s in `#table-body` ; `#row-count` contains `6`
filters by city, case-insensitively	types `LONDON`	1 row, containing *Ada Lovelace*
a non-matching filter shows no rows	types `zzzz`	0 rows
sorts by name A-Z	clicks **Sort by name**	the first row's name is *Ada Lovelace*
sorts by age ascending	clicks **Sort by age**	the first row's name is *Margaret Hamilton* (33)

New ideas

1. Objects

An **object** groups named values (its **properties**) in curly braces. An array of objects is how almost all real data looks, from a list of products to a JSON reply from a server:

JAVASCRIPT an array of objects

```
const people = [
  { name: 'Grace', age: 45 },
  { name: 'Ada', age: 36 },
  { name: 'Alan', age: 41 },
];
console.log(people[1].name, people[1].age);
```

OUTPUT

```
Ada 36
```

`people[1]` is the second object; `.name` reads one property from it.

C++ Corner

An object here is like a `struct Person { std::string name; int age; };`, written as a value on the spot, with no type declared first.

2. Filtering on two fields

The filter keeps an employee if the query is in the name **or** in the city. `||` means *or*: it's true if either side is true. When the test takes more than one line, the arrow function needs `{ }` and a `return`:

JAVASCRIPT a filter test on two fields

```
const rows = employees.filter((emp) => {
  const name = emp.name.toLowerCase();
  const city = emp.city.toLowerCase();
  return name.includes(query) || city.includes(query);
});
```

As on Day 4, an empty query is included in every name, so all rows are kept.

3. Sorting: the trap and the fix

JAVASCRIPT sort() without and with a compare function

```
const nums = [10, 9, 100, 1];
console.log([...nums].sort());
console.log([...nums].sort((a, b) => a - b));
```

OUTPUT

```
[ 1, 10, 100, 9 ] ← sorted as TEXT: "10" < "9"
[ 1, 9, 10, 100 ] ← sorted as numbers
```

Without arguments, `sort()` compares items **as text**, letter by letter, so `'100'` comes before `'9'` (because `'1'` comes before `'9'`). Give it a **compare function** `(a, b) => ...` that returns:

- a **negative** number if `a` should come first,
- a **positive** number if `b` should come first,
- **0** if they're equal.

For numbers, `a - b` does exactly that (ascending). For text, use `localeCompare`:

JAVASCRIPT localeCompare returns -1, 1 or 0

```
console.log('Ada'.localeCompare('Grace'));
console.log('Grace'.localeCompare('Ada'));
console.log('Ada'.localeCompare('Ada'));
```

OUTPUT

```
-1 ← Ada comes first
1 ← Ada (b) comes first
0 ← equal
```

So sorting objects by a property is `(a, b) => a.age - b.age` or `(a, b) => a.name.localeCompare(b.name)`. (`[...nums]` makes a copy so the example's `nums` stays as it was; you don't need it in the exercise, as you'll see next.)

4. `sort` changes the array; `filter` makes a new one

JAVASCRIPT sorting a filtered copy leaves the original alone

```
const copy = people.filter(() => true);
copy.sort((a, b) => a.age - b.age);
console.log(copy.map((p) => p.name));
console.log(people.map((p) => p.name));
```

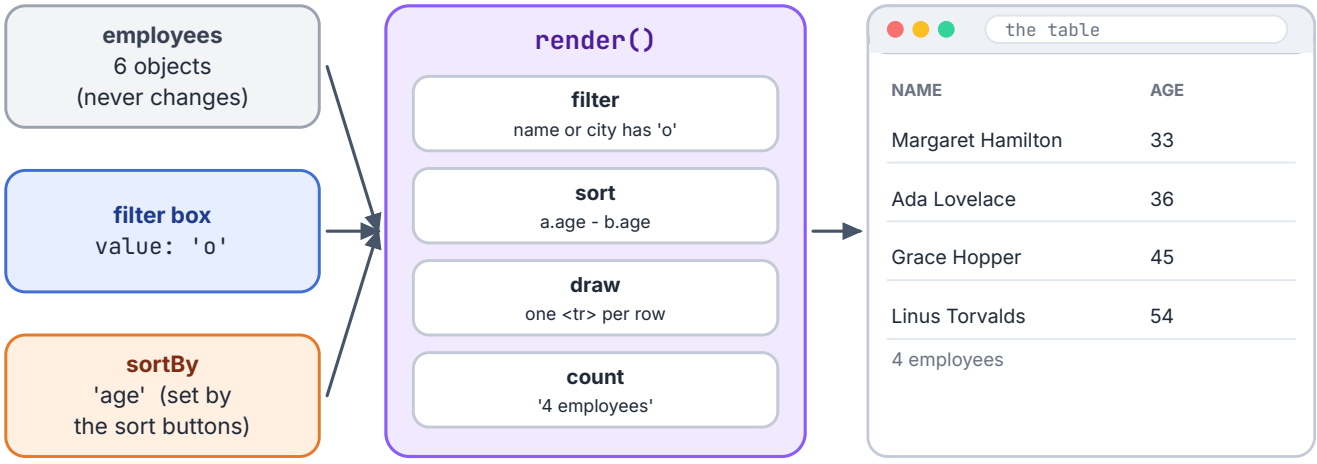
OUTPUT

```
[ 'Ada', 'Alan', 'Grace' ] ← the copy is sorted by age
[ 'Grace', 'Ada', 'Alan' ] ← the original order is unchanged
```

`sort` rearranges the array **in place**. That would be a problem if you sorted `employees` itself, but `filter` always returns a **new** array, so sorting the filtered `rows` leaves the original data untouched. (`map` makes a new array with each item transformed; here it just picks out the names to print.)

5. State + render

The table depends on **two** things the user controls: the filter text and the chosen sort. Keep the sort choice in a variable (the text is already in the input box), and let **one** `render()` build the table from both, every time anything changes:



Every listener only changes the state, then calls render().

Figure 9.1 *Two controls, one `render()`. Each listener only updates the state (or nothing, for the filter, whose state is the input box) and calls `render()`, which always runs the same four steps: filter, sort, draw, count.*

Because `render()` always applies **both** the filter and the sort, they combine for free: filter for `o`, then sort by age, and you get the matching rows in age order.

6. A row from a template

A `<tr>` with three `<td>` cells can be built in one line with a template literal and `innerHTML`:

JAVASCRIPT one table row

```
tr.innerHTML = `<td>${emp.name}</td><td>${emp.age}</td><td>${emp.city}</td>`;
```

This is safe **here** because the data is fixed in the file. If the text came from users, you would create each `<td>` and set its `textContent` (Day 4's XSS warning).

Read the starter files

File	What is there	What you will use
`index.html`	complete	`#filter`, `#sort-name`, `#sort-age`, an empty `<tbody id="table-body">`, `#row-count`
`style.css`	two empty rules	`th` and `td`
`script.js`	the `employees` array (**don't change it**) and comments	everything below it

The plan in plain English

1. Select the five elements.
2. State: `let sortBy = '';` (no sorting yet).
3. `render()`:
    1. The query: the input's text, trimmed and lower-cased.
    2. **Filter** employees whose name or city includes the query.
    3. **Sort** the result if `sortBy` is `'name'` or `'age'`.
    4. **Draw:** empty the `<tbody>`, then one `<tr>` per employee.
    5. **Count:** `6 employees` (or `1 employee`).
4. Listeners: `input` on the filter calls `render`; each sort button sets `sortBy` and calls `render()`.
5. Call `render()` at the end.
6. CSS: `th` gets a background, padding and left-aligned text; `td` gets padding and a bottom border.

> ✅ **Hint 1**
>
> Copy the structure of your Exercise 05 `render()`. The new parts are the second field in the filter, the sort step, and `<tr>` instead of `<li>`.

> ✅ **Hint 2**
>
> Get the drawing working first, with no filter and no sort: `employees.forEach(...)` making rows. Then add the filter, then the sort.

✅ Hint 3

JAVASCRIPT the sort step

```
if (sortBy === 'name') {
  rows.sort((a, b) => a.name.localeCompare(b.name));
} else if (sortBy === 'age') {
  rows.sort((a, b) => a.age - b.age);
}
```

📝 Stop here and build it

Then run `npm test -- 15-dynamic-table`.

The solution, line by line

JAVASCRIPT 15/script.js (below the employees array)

```
const filterInput = document.getElementById('filter');
const sortNameBtn = document.getElementById('sort-name');
const sortAgeBtn = document.getElementById('sort-age');
const tableBody = document.getElementById('table-body');
const rowCount = document.getElementById('row-count');

let sortBy = ''; // '' (no sorting), 'name' or 'age'

function render() {
  const query = filterInput.value.trim().toLowerCase();

  // 1. Filter: keep employees whose name or city contains the query
  const rows = employees.filter((emp) => {
    const name = emp.name.toLowerCase();
    const city = emp.city.toLowerCase();
    return name.includes(query) || city.includes(query);
  });

  // 2. Sort the filtered list
  if (sortBy === 'name') {
    rows.sort((a, b) => a.name.localeCompare(b.name));
  } else if (sortBy === 'age') {
    rows.sort((a, b) => a.age - b.age);
  }

  // 3. Draw one <tr> per employee
  tableBody.innerHTML = '';
  rows.forEach((emp) => {
    const tr = document.createElement('tr');
    tr.innerHTML = `<td>${emp.name}</td><td>${emp.age}</td><td>${emp.city}</td>`;
    tableBody.appendChild(tr);
  });

  // 4. Update the count
  const word = rows.length === 1 ? 'employee' : 'employees';
```

```
  rowCount.textContent = `${rows.length} ${word}`;
}

filterInput.addEventListener('input', render);

sortNameBtn.addEventListener('click', () => {
  sortBy = 'name';
  render();
});

sortAgeBtn.addEventListener('click', () => {
  sortBy = 'age';
  render();
});

render();
```

Lines	What they do
17	The state the user can change with buttons. The empty string means “keep the original order”.
20	The current filter text, read fresh on every render.
23–27	A new array with only the matching employees.
30–34	Sort that new array in place, if a sort was chosen.
37–42	Clear the old rows and draw the new ones.
45–46	The count, with the right word.
49	Typing re-renders.
51–59	Each button changes the state, then re-renders.
61	Draw the table at load: 6 rows before any action.

CSS 15/style.css

```
th {
  text-align: left;
  background: #f8fafc;
  padding: 10px 12px;
  color: #64748b;
}

td {
  padding: 10px 12px;
  border-bottom: 1px solid #e2e8f0;
}
```

The repository’s `solution/` keeps both pieces of state in one object, `const state = { query: '', sort: null };`, and updates `state.query` in the input listener. That shape is common in bigger apps (and in React); for one input and two buttons, both versions are fine.

Mistakes beginners make

⚠️ `rows.sort()` **with no compare function**

Sorting objects without a compare function compares them as text (`"[object Object]"`), so nothing useful happens. Always pass `(a, b) => ...` .

⚠️ `return a.age > b.age` **in the compare function**

It returns `true` / `false` , never a negative number, so `sort` can't tell which item comes first. In Chrome and Node.js it doesn't sort at all: we tried it on the six ages and got them back in their original order. Use `a.age - b.age` .

⚠️ **Sorting inside the button listener only**

If the button sorts and draws, but `render()` (called while typing) doesn't sort, the sort is lost as soon as the user types. Put **all** the steps in `render()` and let the listeners only change the state.

Interview questions

Q1 **How does `sort` decide the order, and why does `[10, 9, 1].sort()` give `[1, 10, 9]` ?**

With no compare function, items are converted to strings and compared character by character, so `'10'` comes before `'9'` . Pass a compare function that returns negative, zero or positive, such as `(a, b) => a - b` for numbers.

Q2 **Does `sort` change the original array?**

Yes, it sorts **in place** and also returns the same array. To keep the original, sort a copy: `[...arr].sort(...)` , `arr.slice().sort(...)` , or, in newer browsers, `arr.toSorted(...)` . `filter` and `map` always return new arrays.

Q3 **What does "state + render" mean, and why is it useful?**

Keep everything the page depends on in a few variables (the **state**) and have one function that rebuilds the view from the state. Every event just updates the state and calls render. Bugs from forgetting to update one part of the page disappear, and this is exactly the idea behind React.

Q4 **How would you sort Z–A, or toggle the direction on a second click?**

Swap the arguments: `b.name.localeCompare(a.name)` . To toggle, keep a `direction` variable (`1` or `-1`), flip it on a second click of the same button, and multiply: `(a, b) => direction * (a.age - b.age)` .

Practice twist

1. Make the column headers clickable for sorting, and remove the two buttons.

2. Make a second click on **Sort by age** sort oldest first (see the last interview answer).
3. Add a fourth column, **Senior**, that shows *Yes* for employees aged 50 or more.

DAY 10 60-minute timed round · three patterns · cheat sheets · interview questions

JS

Mock Round and Revision

TODAY YOU WILL LEARN

- A **mock round** under real conditions: two reported questions and one exercise you have never seen
- The **three patterns** that solve every exercise, on one page
- **Cheat sheets** for the DOM, the JavaScript you used, CSS and HTML
- A **debugging checklist** for when something doesn't work
- **Rapid-fire interview questions** to answer out loud

💻 **Today's plan (2 hours)**

0–10 set up a fresh copy · **10–70** mock round (timer on, no notes) · **70–85** grade and review · **85–120** revision pages and interview questions, out loud.

10.1 The mock round

Set up a fresh copy

Your own folder holds your finished solutions, so make a **second, untouched copy** for the mock round. In the folder **above** `accenture-frontend-practice`:

TERMINAL a clean copy for the mock round

```
git clone https://github.com/dexterhere04/accenture-frontend-practice.git mock-round
cd mock-round
npm install
```

The Playwright browser is already installed from Day 0, so you don't need that step again. Open `mock-round` in VS Code.

The rules

- **60 minutes in total**, with a timer you can see.
- **No** guide, **no** `solution/` folders, **no** internet. The browser console is allowed.
- Read each `task.md` **fully** before you write anything (2 minutes well spent).
- Grade each task as soon as you finish it. In the real round, partial scores count, so a task with 4 of 5 tests passing is worth a lot.

Task	Exercise	Time	Why
A	**05** E-commerce Search Box	25 min	reported from the real round
B	**17** Live Digital Clock	15 min	reported from the real round
C	**12** Progress Bar + Spinner	20 min	**new to you**: the real round won't be an exercise you've practised

For Task C, this card is your "documentation". Real tasks often include a hint like this.

📖 **Documentation card for Task C**

- `transition: width 0.4s ease;` on an element makes any later change of its `width` **animate** over 0.4 seconds instead of jumping.
- In JavaScript, `el.style.width = '100%';` sets the width directly on the element.
- A **keyframe animation** has two parts. Define the movement once, then attach it:

CSS a continuous spin

```css
@keyframes spin {
  to { transform: rotate(360deg); }
}
.thing {
  animation: spin 0.8s linear infinite;   /* name, duration, speed curve, repeat */
}
```

- Give the bar `width: 0` at first, or it starts full (a `<div>` is as wide as its parent by default).

After the round

Run the grader on all three (`npm test -- 05-ecommerce` , `npm test -- 17-live` , `npm test -- 12-progress`). Then, for every test that failed or every minute you lost, write one line: *what slowed me down?* Common answers, and the fix:

What slowed you down	Fix
Didn't know where to start	Always the same first lines: select every id the task mentions. Then the listener. Then `render()` or the toggle.
A typo in an id or class	Copy ids from `index.html` instead of typing them.
Forgot to call `render()` / `updateClock()` at load	Add it to your personal checklist: *"Does the page look right before any click?"*
Spent long on CSS polish	Do the JavaScript first: most tests check behaviour. Style last.

10.2 The three patterns

Every exercise in this guide is one of these, built on **select → listen → change**.

Pattern 1 · Toggle

One element, two states: exercises **08** and **09** (and bonus 10). The state lives in the page itself (a class, or `input.type`), and CSS decides how each state looks.

Pattern 2 · Render from data

A list drawn from an array: exercises **05** and **15** (and bonus 19, 20). One `render()` rebuilds everything; call it at load and after every change.

Pattern 3 · All off, one on

Many elements, one active: exercises **13**, **14** and **16**. Read the old state first if a second click should close it (13).

JAVASCRIPT pattern 1 · toggle

```
btn.addEventListener('click', () => {
  const on = el.classList.toggle('open');
  btn.setAttribute('aria-expanded', String(on));
});
```

JAVASCRIPT pattern 2 · render from data

```
function render() {
  const query = input.value.trim().toLowerCase();
  const items = data.filter((item) => item.toLowerCase().includes(query));
  list.innerHTML = '';
  items.forEach((item) => {
    const li = document.createElement('li');
    li.textContent = item;
    list.appendChild(li);
  });
}
input.addEventListener('input', render);
render();
```

JAVASCRIPT pattern 3 · all off, one on

```
items.forEach((item) => {
  item.addEventListener('click', () => {
    items.forEach((x) => x.classList.remove('active'));
    item.classList.add('active');
  });
});
```

10.3 Cheat sheets

The DOM

Goal	Code
one element by id	`document.getElementById('count')` (no `#`)
first match of a CSS selector	`document.querySelector('.tab.active')`
all matches	`document.querySelectorAll('.tab')` → loop with `forEach`
search inside an element	`item.querySelector('.icon')`
listen	`el.addEventListener('click', fn)` · events: `click`, `input`, `submit`, `keydown`
stop the default action	`event.preventDefault()` (forms, links)
read typed text	`input.value`
set text	`el.textContent = 'Hi'`
set HTML (trusted text only) / empty	`el.innerHTML = '<b>Hi</b>'` · `el.innerHTML = ''`
classes	`el.classList.add / remove / toggle / contains('x')`
attributes	`el.setAttribute('aria-pressed', 'true')` · `el.getAttribute('x')`
`data-*`	`el.dataset.tab` reads `data-tab`
show / hide	`el.hidden = true / false`
create and attach	`document.createElement('li')` → fill → `parent.appendChild(li)`
remove	`el.remove()`
count children	`list.children.length`

JavaScript you used

Topic	Code
variables	`const` by default, `let` when it changes
compare	`===`, `!==`, `<`, `>=`; combine with `&&` (and), `||` (or), `!` (not)
decide	`if (...) { } else if (...) { } else { }` · `cond ? a : b`
functions	`function name(x) { return x; }` · `(x) => { ... }` · `(x) => x * 2`
strings	`.trim()` `.toLowerCase()` `.includes('a')` `.length` `.padStart(2, '0')` · `` `Hi ${name}` ``
numbers	`Number('2')` · `String(7)` · `a % b` (remainder)
arrays	`.length` `[0]` `.forEach(fn)` `.filter(fn)` `.map(fn)` `.sort((a, b) => a - b)`
objects	`{ name: 'Ada', age: 36 }` · `emp.name`
time	`new Date()` · `.getHours()` · `setInterval(fn, 1000)` · `setTimeout(fn, ms)`
regex	`/^\S+@\S+\.\S+$/.test(text)`

CSS

Goal	Code
select	`tag` · `.class` · `#id` · `.a .b` (b inside a) · `.a.b` (both) · `:hover` `:focus` · `[data-x="y"]`
who wins	id 100 · class / pseudo-class 10 · tag 1; equal → the later rule
box	`padding` (inside) · `border` · `margin` (outside) · `box-sizing: border-box`
row	`display: flex; gap; justify-content; align-items`
grid	`display: grid; place-items: center` · `grid-template-columns: repeat(auto-fill, minmax(220px, 1fr))`
center	flex: `justify-content: center; align-items: center` · grid: `place-items: center` · absolute: `top/left: 50%; transform: translate(-50%, -50%)`
hide	`display: none` (no space) · `visibility: hidden` (keeps space)
looks	`border-radius` · `box-shadow: 0 10px 30px rgba(0,0,0,.12)` · `cursor: pointer` · `font: inherit`
variables	`:root { --bg: #fff; }` · `background: var(--bg)`
motion	`transition: transform 0.2s` · `@keyframes` + `animation`
responsive	`@media (max-width: 600px) { ... }`

HTML

Element / attribute	Use
`<label for="id">`	names an input; `for` = the input's `id`
`<input type="email / tel / password / text">`	the right keyboard, check or masking
`required`	a boolean attribute: write the bare word
`<fieldset>` + `<legend>`	group related fields with a title
`<button type="button">`	a normal button; inside a form, the default type is `submit`
`aria-pressed`, `aria-expanded`, `aria-selected`	tell screen readers the state; keep them in sync
`data-*`	your own information on an element
`<script src>` before `</body>`	the elements exist when the script runs

10.4 When something doesn't work

Go through this list **in order**. Most problems are found by step 3.

1. **Open the console (F12).** Is there red text? Read it: it gives the file and line.
2. `Cannot read properties of null` : a `getElementById` / `querySelector` found nothing. Check the spelling against `index.html`, and remember: no `#` in `getElementById`, a `#` or `.` **is** needed in `querySelector`.
3. **A click does nothing and there's no error:** put `console.log('clicked')` as the first line of the listener. Nothing printed? The listener isn't attached (wrong element, or brackets after the function name). Printed? The problem is inside the listener: log the values you use.
4. **Works once, then goes wrong:** state isn't stored (Day 8's `index`), a list isn't cleared before redrawing (Day 4), or an old message isn't cleared (Day 6).
5. **CSS has no effect:** a typo in the selector, a missing `;` on the line above, or a more specific rule winning. Right-click the element → **Inspect**: the **Styles** panel shows which rules apply and crosses out the ones that lose.
6. **The page looks right but a test fails:** read *Expected* / *Received* and the locator. The grader looks for **exact** ids, classes and attribute values (`'true'`, not `true` in some other place; `completed` on the `<li>`, not the `<span>`).

10.5 Rapid-fire interview questions

Cover the answers, say yours out loud in under 30 seconds, then check.

Q1 What happens when the browser loads a page?

It reads the HTML from top to bottom and builds the DOM, loads the CSS to style it, runs each `<script>` when it reaches it, and draws the page. Afterwards, JavaScript reacts to events and changes the DOM, and the browser redraws.

Q2 What is the DOM?

The tree of objects the browser builds from the HTML, which JavaScript reads and changes.

Q3 `let`, `const`, `var`?

`const` can't be reassigned, `let` can; both are block-scoped. `var` is function-scoped and hoisted. Prefer `const`, then `let`.

Q4 `==` versus `===`?

`===` compares without converting types; `==` converts first (`5 == '5'` is `true`). Use `===`.

Q5 `textContent` versus `innerHTML`?

`textContent` sets plain text; `innerHTML` parses HTML, which is a security risk (XSS) with user input.

Q6 What is an event listener? What is a callback?

`addEventListener(type, fn)` registers `fn` to be called when the event happens. A callback is a function you pass to be called later, like `fn` here or the function given to `setInterval`.

Q7 What does `preventDefault()` do?

Cancels the browser's default action: a form submit's page reload, a link's navigation.

Q8 What is event bubbling, and what is event delegation?

An event on an element also fires on each of its ancestors, up to the document (**bubbling**). **Delegation** uses this: one listener on a parent handles events from many children, using `event.target` to see which child was clicked. Useful for long or changing lists.

Q9 What is a closure?

A function that remembers variables from where it was created, like each to-do's Delete listener remembering its `li`.

Q10 `forEach`, `map`, `filter`?

`forEach` runs a function per item and returns nothing; `map` returns a new array of transformed items; `filter` returns a new array of the items that pass a test.

Q11 Why does `[10, 9, 1].sort()` give `[1, 10, 9]`?

Without a compare function, items are compared as strings. Use `(a, b) => a - b`.

Q12 `setTimeout` versus `setInterval`?

Once after a delay, versus repeatedly. Both can be cancelled with their id.

Q13 What is the box model?

Content, padding, border, margin; `box-sizing: border-box` makes `width` include padding and border.

Q14 Flexbox or grid?

Flexbox for one direction (a row or a column), grid for two (rows and columns).

Q15 How do you center a div?

`display: flex; justify-content: center; align-items: center;` on the parent, or `display: grid; place-items: center;`.

Q16 `display: none` versus `visibility: hidden`?

`none` removes it from the layout; `hidden` hides it but keeps its space.

Q17 What is CSS specificity?

The scoring that decides which rule wins: ids > classes and pseudo-classes > elements; ties go to the later rule.

Q18 What are CSS variables?

Custom properties (`--bg`) read with `var(--bg)`. They inherit and can be overridden per element or class, which is ideal for themes.

Q19 What is semantic HTML, and what is ARIA?

Semantic HTML uses elements for their meaning (`button`, `label`, `nav`). ARIA attributes add information screen readers can't get from HTML alone, such as `aria-expanded`.

Q20 What is `localStorage`?

A small key-value store in the browser that survives reloads. Values are strings, so objects are saved with `JSON.stringify` and read back with `JSON.parse`. (Bonus 19.)

Q21 What is `fetch`, and what do `async` / `await` do?

`fetch(url)` requests data and returns a **Promise**. `await` pauses an `async` function until the Promise settles, so asynchronous code reads like normal code; wrap it in `try` / `catch` to handle errors. (Bonus 18; Chapters 5 and 7 of your JavaScript notes.)

Q22 Walk me through your search box.

Select the input, list, count and empty message. One `render()` function lower-cases the query, filters the products with `includes`, clears the list, creates an `<li>` per match, updates the count and shows the empty message when nothing matches. It runs at load and on every `input` event.

10.6 After Day 10

You have done 14 exercises, each with tests passing. To keep it fresh until the assessment:

- **Every other day:** redo one exercise from a fresh copy, timed. Rotate through 05, 17, 06, 15, 13.
- **The six bonus exercises** in the next chapter, in the order given. Each one adds one new idea to what you know.
- Before the real round, read 10.2 and 10.4 again. They fit on two pages for a reason.

BONUS After Day 10 · one new idea each

JS

The Other Six Exercises

TODAY YOU WILL LEARN

- **10** Responsive Navbar: media queries
- **11** Responsive Card Grid: `auto-fill` and `minmax`, hover lifts
- **12** Progress Bar + Spinner: transitions and `@keyframes` (your mock-round Task C)
- **19** Persistent Notes: saving to `localStorage` with JSON
- **20** Quiz: several pieces of state and step-by-step screens
- **18** Fetch & Render: loading data with `fetch`, `async` / `await` and errors

Each of these adds **one** idea to what you already know. They're in the order to do them. For each one: what's new, the plan, and the hint that unlocks it. There's no line-by-line solution here; when you're done, compare with the exercise's `solution/` folder, which passes all its tests.

Exercise 10 · Responsive Navbar

Pattern: toggle (Day 2 and 5), plus **media queries**.

A **media query** applies CSS only when a condition is true, usually the screen width:

CSS from exercises/10-responsive-navbar/solution/style.css (part)

```css
@media (max-width: 600px) {
  .nav-toggle {
    display: inline-block;
  }

  .nav-links {
    display: none;
    width: 100%;
    flex-direction: column;
    align-items: flex-start;
    gap: 14px;
  }

  .nav-links.open {
    display: flex;
  }
}
```

Plan: on desktop, `.nav-links` is a flex row and `.nav-toggle` has `display: none`. Inside the media query: show the toggle, hide the links, and show them as a **column** when they have the class `open`. The JavaScript is Exercise 09 again: toggle `open` on `#nav-links` and keep `aria-expanded` in sync.

Hint: test it by making the browser window narrow, or open DevTools (F12) and click the phone icon (**device toolbar**) to choose a width.

Exercise 11 · Responsive Card Grid

Pattern: CSS only. **New:** a grid that picks its own number of columns.

CSS from exercises/11-responsive-card-grid/solution/style.css (part)

```css
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
  gap: 16px;
}
```

Read it as: *"make as many columns as fit (`auto-fill`), each at least 220px and at most an equal share of the space (`minmax(220px, 1fr)`)."* On a wide screen that's 4 or 5 columns, on a phone 1, with **no media query**. `1fr` means *one fraction of the free space*.

Plan: the grid rule above; on `.card`, a background, padding, radius, shadow and `transition: transform 0.18s ease, box-shadow 0.18s ease;`; on `.card:hover`, `transform: translateY(-4px)` and a stronger shadow.

Hint: `transition` goes on the **normal** state (`.card`), not on `:hover`, so the card animates both up and back down.

Exercise 12 · Progress Bar + Spinner

Pattern: toggle-like (one click, three changes). **New:** `transition`, `@keyframes`, `el.style`.

You met this in the mock round with the documentation card (10.1). If it didn't pass then, do it now: `.progress-bar` gets `height: 100%`, `width: 0`, a colour and `transition: width 0.4s ease`; `.spinner` gets `animation: spin 0.8s linear infinite`; define `@keyframes spin { to { transform: rotate(360deg); } }`. On click of `#start`: `bar.style.width = '100%'`, the label `'100%'`, and `setAttribute('aria-valuenow', '100')` on the `.progress` track.

Exercise 19 · Persistent Notes

Pattern: render from data (Day 4), with the data in an **array you change**. **New:** `localStorage` and JSON.

`localStorage` is a small store in the browser that **survives a reload**. It only stores **strings**, so an array of objects is turned into a string (**JSON**) to save it and parsed back to load it:

JAVASCRIPT from exercises/19-persistent-todo/solution/script.js (part)

```
function loadNotes() {
  try {
    const raw = localStorage.getItem(STORAGE_KEY);
    return raw ? JSON.parse(raw) : [];
  } catch (error) {
    return [];
  }
}

function saveNotes() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(notes));
}
```

`STORAGE_KEY` is a `const` at the top of the file: `'accenture-notes'`, the name the data is saved under. `getItem` returns `null` the first time (nothing saved yet), hence `raw ? ... : []`. The `try` / `catch` handles the rare case of broken saved data.

Plan:

1. `let notes = loadNotes();`, an array of `{ id, text }`.
2. `render()`: clear the list; for each note, create `<li class="note">` with a `<span>` and a `<button class="note-delete">`; hide or show `#empty`.
3. **Add:** on submit, guard against empty text, `notes.push({ id: `${Date.now()}-${Math.random()}`, text })` (the time plus a random number, so two notes never share an id), `saveNotes()`, clear the input, `render()`.
4. **Delete:** `notes = notes.filter((n) => n.id !== note.id)`, then save and render.
5. **Clear all:** `notes = []`, save, render. And call `render()` once at load.

Hint: the rule is **change the array → save → render**, every time. Unlike Exercise 06, you never remove an `<li>` directly; the array is the truth and the page is redrawn from it. Open DevTools → **Application** → **Local Storage** to watch the saved string change.

Exercise 20 · Quiz

Pattern: render from data, with **three pieces of state**: which question (`current`), the `score`, and which option is `selected`.

Plan:

1. `let current = 0; let score = 0; let selected = null;`
2. `renderQuestion()`: reset `selected` to `null` and disable Next (`nextBtn.disabled = true`); show `Question ${current + 1} of ${questions.length}` and the question text; clear `#options`; create a `<button class="option">` per option. Each option's click sets `selected` to

its index, gives **only** that button the class `selected` (all off, one on!), and sets `nextBtn.disabled = false`.

3. **Next:** if `selected === questions[current].answer`, add 1 to `score`. Then `current += 1`; if there's another question, `renderQuestion()`, otherwise `showScore()`.
4. `showScore()`: `quizEl.hidden = true; scoreEl.hidden = false;` and the text `You scored ${score} / ${questions.length}`.

Hint: `forEach((option, index) => ...)` gives you the **position** as the second argument; that's the number to compare with `answer`.

Exercise 18 · Fetch & Render

Pattern: render from data, where the data arrives **later**, from a file. **New:** `fetch`, `async` / `await`, `try` / `catch`. Chapters 5 and 7 of your JavaScript notes explain Promises and `async` / `await` in depth.

JAVASCRIPT from exercises/18-fetch-and-render/solution/script.js (shortened)

```
try {
  const response = await fetch('./data.json');
  if (!response.ok) {
    throw new Error(`Request failed with status ${response.status}`);
  }
  const users = await response.json();
  // ... create one <li class="user"> per user
  statusEl.textContent = '';
} catch (error) {
  statusEl.textContent = 'Something went wrong while loading users.';
}
```

- `fetch` returns a **Promise**: a value that arrives later. `await` waits for it, and only works inside a function marked `async`.
- `fetch` doesn't fail on a 404 or 500 error. Check `response.ok` and `throw` yourself, so the `catch` shows the error message. The grader tests exactly this by making the server answer 500.
- Show `Loading...` in `#status` **before** the `fetch`, and clear it after rendering.

⚠️ **Opening exercise 18 by double-clicking `index.html`**

Browsers block `fetch` from pages opened as files. We tried it: the console says *Access to fetch at 'file:///.../data.json' from origin 'null' has been blocked by CORS policy*, and the page shows the error message. Run `npm run dashboard` and open the exercise through **http://localhost:4173** instead. (The grader uses its own server, so the tests work either way.)